

MASTER THESIS

Learning-augmented Models for Parking Permit and Related Problems

vorgelegt von

Marcus Philip Rottschäfer

MIN-Fakultät

Fachbereich Informatik

Gruppe Theorie Effizienter Algorithmen (TEA)

Studiengang: Informatik

Matrikelnummer: 7327475

Abgabedatum: 05.02.2023

Erstgutachter: Prof. Dr. Peter Kling

Zweitgutachterin: Prof. Dr. Petra Berenbrink

Acronyms

CR Competitive Ratio

ML Machine Learning

LP Linear Program

Contents

1	Introduction	1
2	Preliminaries	3
2.1	Online and Offline Computation	3
2.2	Competitive Analysis	4
2.2.1	Competitive Analysis for an Online Ski Rental Strategy	5
2.2.2	Competitive Ratio for Randomized Algorithms	5
2.3	Learning-augmented Online Algorithms	7
2.3.1	Example: Learning-augmented Ski Rental	8
2.4	Primal-Dual Methods for Online Algorithm Design	9
2.4.1	Linear Programming and Duality	10
2.4.2	Duality Theorems and Approximate Complementary Slackness	11
2.4.3	Randomized Rounding	12
2.4.4	Method of Dual Fitting	13
2.4.5	Primal-Dual Scheme	14
2.5	Problem Sets	15
2.5.1	Parking Permit	15
2.5.2	Weighted Set Cover	17
2.5.3	Online Facility Location	18
2.5.4	Online Facility Leasing	20
3	Related Work	23
4	Learning-augmented Parking Permit	27
4.1	Parking Permit as Weighted Set Cover	27
4.2	Predictions for Parking Permit	28
4.3	Deterministic, Learning-augmented Parking Permit	29
4.3.1	Analysis of DETLAPP	30
4.4	A Randomized, Learning-augmented Algorithm for Parking Permit	33
4.4.1	Analysis of LAPP	34
5	Learning-augmented Facility Leasing	39
5.1	Predictions for Facility Leasing	39
5.2	A Learning-augmented Algorithm for Facility Leasing	40
5.2.1	Analysis of LAFL	42
6	Conclusion	49

1 Introduction

In an attempt to act under an uncertain future, one can assume that nothing about the future can be known. Online algorithms follow this assumption by making decisions, acting upon a part of an input sequence being revealed at each time step, absent from any predictions about the future distribution of the input. In fact, in online algorithm design, the design goal is to be as good as possible under the worst possible future to come. The quantity of "goodness" is measured by comparing the objective value of the online algorithm (often called the cost) to the best objective value possible assuming full knowledge of the future. This is the objective value of the optimal offline algorithm acting on the full input sequence. Following Borodin and El-Yaniv [BEY05], we refer to the maximum of this ratio under all instances as the Competitive Ratio (CR) of the given online algorithm.

Now when designing algorithms for an online problem, our goal of minimizing the CR shifts the design effort to deal with the worst possible instances. This is the most secure design for all possible futures, but, like all worst-case analysis approaches, might suffer from a too pessimistic result compared to the input distributions encountered in practice. For real-world online problems like caching, buying parking permits, or scheduling ride-sharing services, the assumption of an adversary providing the most malicious input sequence turns out to be too unrealistic. Hence including knowledge about the distribution of sequences normally encountered as input in applications would make the analysis more realistic. Unfortunately, this often turns out to be too complex or to require too many simplifying assumptions, even for most simple algorithms.

Now the whole field of Machine Learning (ML) provides excellent tools for the task of capturing (learning) the distribution of sampled data, aimed at generalizing and predicting suitable characteristics of future input data. This enables another take on decision making under uncertain future by deciding based on the model's current predictions about the future. While ML often gives compelling predictive performance on real-world tasks (e.g stock price prediction [PSTK15], predicting the next action in a strategic game under imperfect information [VBC⁺19], or predicting the next token in sentences [BMR⁺20]), this framework misses the robustness and worst-case guarantees provided by classical algorithm design.

The field of learning-augmented online algorithms now tries to achieve the best of both worlds by using the predictive power and distribution learning capabilities of ML models to inform an online algorithm about the distribution of data it is likely to encounter. The main goal here is to give the online algorithm cues about the future in the form of (i.e. machine-learned) predictions, allowing it to achieve better objective values on more realistic inputs, while retaining the robustness and worst-case guarantees for bad inputs. If provided a good guess of what might be the next input, or what action should be taken with the likely future input in consideration, an online algorithm provided this learning-augmentation in the form of predictions can improve its decision making, leading to better solutions.

1 Introduction

This new design paradigm leads to online algorithms with CRs now additionally depending on the quality of the ML prediction, which we often express as a properly defined prediction error η . The CR should still account for a worst-case bound (under the assumption of a large prediction error η), matching the CR of the best pure online algorithm. In the framework of learning-augmented algorithms, this property is referred to as robustness. Further, a learning-augmented algorithms CR should be a function of η gradually decreasing with decreasing η , yielding a CR of 1 for perfect predictions $\eta = 0$. We call this property consistency and this reflects the improvement in the CR over the classical, non-learning-augmented online algorithms, given good predictions.

In this thesis, we present two learning-augmented online algorithms for problems requiring to find some sort of *covering* of the input. The first problem is the Parking Permit problem introduced by Meyerson [Mey05] and it asks to find the optimal (cheapest) covering of an online-revealed sequence of driving days and non-driving days, choosing between K different parking permits purchasable each with different cost and different duration of validity. Informally, not knowing what the future driving schedule looks like, it asks which kind of permit $\in \{\text{DAY, WEEK, MONTH, YEAR}\}$ to buy. We give a randomized learning-augmented algorithm, parameterized by a user-defined $\lambda \in [0, 1]$, based on the Primal-Dual formulation of the problem. We proof that it achieves a CR of expected $\mathcal{O}(\log \frac{K}{\lambda})$ in any case (robustness) and expected $\mathcal{O}\left(\frac{K}{\lambda(1-K)+K} \cdot \frac{\eta + \text{OPT}}{\text{OPT}}\right)$ for a prediction error η measuring the absolute difference between the optimal schedule cost (OPT) and the cost of following the prediction blindly. In the robustness case, this attains the best-possible CR up to a constant term $-\log \lambda$, while optimal schedules are achievable for $\lambda = 0$ and perfect predictions.

As a second result, we present a deterministic learning-augmented algorithm for the Online Facility Leasing problem. This problem, introduced by Anthony et al. [AG07] in the context of leasing problems, is an extension of the NP-hard Facility Location problem. It further extends Parking Permit by a geometric optimization aspect. Given a set of points in a metric space (requests, revealed in an online-fashion), the goal is to serve the points by opening *facilities*, choosing from a subset of possible facility locations, where a facility can be temporarily opened by choosing between K different *lease* types for that facility. One has to choose the facility locations and lease types as to minimize the total leasing cost plus the total connection cost for a request connecting to the closest currently open facility. We show our algorithm to be $\mathcal{O}(\frac{K}{\lambda} \log D_{\max})$ -robust, where $D_{\max}$ denotes the maximal lease duration among all K lease types. We analyse the algorithm in the Primal-Dual framework. Unfortunately, because of the limited time for this thesis, we were not able to prove an error-dependent upper-bound on the CR yet. As a preparatory step, we highlight a few ideas for the proof and argue about possible error measures.

In Chapter 2 and 3, we introduce the relevant background material and Related Work. Chapter 4 states the learning-augmented algorithm for Parking Permit and a deterministic version due to Schneider [Sch22]. Successively, Chapter 5 states the results for the Online Facility Leasing problem and we summarize our findings in Chapter 6.

2 Preliminaries

This chapter reviews the preliminary background material necessary to understand the context and the analysis of the algorithms presented in this thesis. Formal definitions are provided where useful and most of the notation used throughout the rest of the document is introduced. We begin by reintroducing the concept of online computation.

2.1 Online and Offline Computation

Assuming a sense of purpose, the goal of computation is to find a solution for an optimization problem $\mathcal{P}$. Such a problem is defined in terms of an input set $\mathcal{I}$, a set of feasible outputs for an input $F(I)$ and an objective function $C : \mathcal{I} \times F(\mathcal{I}) \rightarrow \mathbb{R}$ quantifying the goodness of a feasible output relative to an input. A classical (offline) algorithm A for $\mathcal{P}$ takes a whole input $I \in \mathcal{I}$ and outputs a feasible solution $O \in F(I)$ at cost $A(I) := C(I, O)$ [BEY05].

As an example for an optimization problem, we may model a ski trip as an input $I = (n, b)$ where $n \in \mathbb{N}$ encodes the number of days we ski and $b \in \mathbb{N}$ encodes the amount of money for buying new skis. We can decide to buy skis at any day $\leq n$, giving us free skis for the rest of the trip. We can rent skis for one day at cost 1. A feasible output $O \in F((n, b))$ is any combination of buying and/or renting decisions per day that gives us skis for all n days. As an objective $C(I, O)$ to be minimized, we consider the amount of money spent for each buying/renting schedule O . We call this problem offline Ski Rental and use it as a toy example throughout the next sections.

Now among all correct offline algorithms for a problem, we call one OPT (optimal), when for all $I \in \mathcal{I}$ it finds the minimal-cost feasible solution [BEY05, p. 2], satisfying:

$$\text{OPT}(I) = \min_{O \in F(I)} C(I, O) \quad (2.1.1)$$

For example, OPT for offline Ski Rental will buy at day 1 if $n \geq b$ and rent every day else. It can be easily verified that this is the optimal strategy.

Solving problems in this offline scheme is static with respect to the input. It turns out that for many artificial and real-world problems, the introduction of some temporal dynamic into the input makes the model more realistic. For example, consider an elevator control unit. The unit can't wait for 100 requests and then compute the optimal schedule, it needs to send the elevator right when a person calls for it. To some problems like caching or investment planning, this temporal notion is so intrinsic that offline computation is just pointless [BEY05]. Consequently, we have an algorithm running indefinitely and require it to act on incoming pieces of input revealed to the algorithm over time. These pieces of input are well defined.

2 Preliminaries

We call a problem that reveals its input piece-wise in this temporal fashion an online problem, and an algorithm for this problem an online algorithm.

Thus the problems input transforms into a sequence of elementary items $I = (\sigma_1, \dots, \sigma_n)$, each of which needs to be processed and acted on according to the problem constraints before revealing the next item in line. We call an elementary item in this setting a request. The online algorithms input thus reduces from the full I to single requests, where request σ_i is revealed before σ_{i+1} and whenever a new request σ_i is revealed to the infinitely running algorithm, it must have produced a feasible output for $(\sigma_1, \dots, \sigma_{i-1})$.

In the classical online model, the future is unknown and decisions made by an online algorithm ALG are irrevocable. ALG can't change its actions for already processed requests when a new request appears that causes past actions to ideally be changed. Different relaxations to this model exist. In the recourse model, online algorithms have a budget allowing to change past decisions (e.g. [MSVW16, MN20]). The relaxation of semi-online algorithms allows a small look-ahead into future requests or into parts of the problem unknown to the classical online algorithm (e.g. [AH12, KPS⁺18]). Here we focus on the classical online model, but later augment this using a stochastic relaxation based on ML.

Returning to the Ski Rental example, in the online version of the problem, the input parameter n is unknown. The input sequence consists of a series of ones, each encoding the need for a ski on that day, followed by zeros for the rest of the sequence. An online ALG for the problem needs to decide for an incoming one whether to rent or to buy, but does not know whether the next request will be a ski day, or a zero.

2.2 Competitive Analysis

Naturally, an offline algorithm that knows the full input $(\sigma_1, \dots, \sigma_n)$, equivalent to it correctly foresees all the future elements right from the start, is in advantage compared to an online algorithm that cannot revoke its decisions for $(\sigma_1, \dots, \sigma_i)$ when seeing σ_{i+1} . Competitive Analysis is a framework for analysing online algorithms that focuses on this knowledge-based disadvantage of an online strategy compared to an offline strategy. Following Borodin and El-Yaniv [BEY05, p. 3] with the cost of an algorithm on an input sequence denoted by $\text{ALG}(I)$, we define our main performance measure in this framework as

Definition 2.2.1 (c-Competitiveness). *Given an online optimization problem $\mathcal{P}$ and an optimal offline algorithm OPT , we say an online algorithm ALG for $\mathcal{P}$ is **c-competitive**, if there exists a constant α independent of the problem parameters such that for all $I \in \mathcal{I}$:*

$$\frac{\text{ALG}(I)}{\text{OPT}(I)} \leq c + \frac{\alpha}{\text{OPT}(I)}$$

*The infimum over all values c such that the constraint holds is called the **Competitive Ratio (CR)** of an algorithm.*

Thus the CR is a measure of how much more a solution produced by ALG costs compared to the best-possible solution produced with full knowledge of the future request sequence. Because the ratio holds among all input sequences, the CR considers the worst-case cost

factor and hence we count Competitive Analysis to the worst-case analysis approaches. Note that $\frac{\alpha}{\text{OPT}(I)}$ vanishes for increasing input sizes and hence is asymptotically not important for the approximation ratio. For $\alpha \leq 0$, we talk about **strict c-competitiveness**. Further, c may well depend on the problem parameters.

The first application of Competitive Analysis to an online algorithm is due to Sleator and Tarjan [ST85], who showed that the MOVE-TO-FRONT algorithm for the List Accessing problem is strictly 2-competitive, using a potential-function argument. In general, one needs to show that the cost ratio holds for all request sequences, while often it is not exactly clear what OPT does and its cost must be lower-bounded. In this framework, both OPT and ALG may have non-polynomial runtime and in Section 5.2, we will see an example of an online approximation to a NP-hard problem called Facility Leasing.

2.2.1 Competitive Analysis for an Online Ski Rental Strategy

Exemplarily we analyse an algorithm for the (online) Ski Rental problem using the framework of Competitive Analysis. Online algorithm 2.1 rents for $b - 1$ days and then buys the skis.

Algorithm 2.1 BRE (BREAK-EVEN)

Input: σ_i
 if $\sigma_i = 0$ **or** **skis bought** **then return**
 if $i \leq b - 1$ **then**
 rent skis ▷ increase cost by 1
 else
 buy skis ▷ increase cost by b

Trip length n is known to OPT, but unknown to BRE. For $n \leq b - 1$, both OPT and BRE will rent and hence attain the same cost. For $n > b - 1$, OPT will produce the minimum-cost solution and hence buy at cost b on the first day. Our algorithm BRE will output a solution at cost $(b - 1) + b = 2b - 1$, because it rents for the first $b - 1$ days and then buys. Hence the CR of BRE is $\frac{2b-1}{b} \leq 2$. We further note that among all online strategies for Ski Rental, BRE attains the minimal CR (e.g. [KMMO94, PSK18]).

2.2.2 Competitive Ratio for Randomized Algorithms

For randomized algorithms, the algorithms actions and cost turns into a probability distribution. Hence we need to adapt our definition of competitiveness. In order to understand the adapted and extended definition, we first introduce a different viewpoint on Competitive Analysis. The rest of the section is inspired by [BEY05, pp. 3-4,44-46].

Competitive Analysis viewed from Game Theory

The CR is based on two acting entities ALG and OPT. As we seek to find the maximal ratio between the two costs and ALG is fed requests piece-by-piece, we can interpret the Competitive Analysis as a game in which an adversary $\mathcal{ADV}$ at each time step provides

2 Preliminaries

ALG with an input σ_i that maximizes $\text{ALG}(I)$, while minimizing $\text{OPT}(I)$. In this view of constructing a worst-case input sequence, we talk about an adversary providing the (malicious) input piece-by-piece to the online algorithm.

While for a deterministic ALG this view does not provide any new insight, it enables us to more precisely analyse randomized online algorithms by considering different adversaries. It is needed to define an adversary model in the randomized case because it is not clear anymore, what a bad request for ALG is when it acts on it unpredictably based on some distribution. We now give three adversary models considering randomization which differ in the extend to what they know about the randomness, and how they could act on it.

1. **Oblivious Adversary ($\mathcal{OBL}$)** An oblivious adversary is blind to the outcomes of the random choices of ALG. It knows all ALG's code and distributions, but can only provide malicious input generated ahead of the game. This adversary is the weakest and as the input I is available from the start, it is equivalent to OPT.
2. **Adaptive-Offline ($\mathcal{AD} - \mathcal{OFF}$)** This adversary can adapt to the actions taken by ALG, meaning it is allowed to generate the malicious request sequence online, knowing the outcomes of every random choice of ALG. This adversary's cost is measured as the offline optimal cost on the generated sequence. It is the most powerful among the three adversaries.
3. **Adaptive-Online ($\mathcal{AD} - \mathcal{ON}$)** As the $\mathcal{AD} - \mathcal{OFF}$ model, this adversary knows about all outcomes of ALG and can adapt its strategy. But we require that this model pays the cost of serving the generated request sequence in an online-fashion, too. This makes it an intermediate-level type.

Knowing about different adversaries, we now redefine competitiveness for randomized algorithms under this game-theoretic notion.

Definition 2.2.2 (c-Competitiveness against Adversaries). *Given an online optimization problem $\mathcal{P}$ and an adversary $\mathcal{ADV} \in \{\mathcal{OBL}, \mathcal{AD} - \mathcal{OFF}, \mathcal{AD} - \mathcal{ON}\}$, we say a randomized online algorithm RAND for $\mathcal{P}$ is **c-competitive** against $\mathcal{ADV}$, if there exists a constant α independent of the problem parameters such that for all inputs I :*

$$\mathbb{E} [\text{RAND}(I) - \mathbf{c} \cdot \mathcal{ADV}(I)] \leq \alpha$$

The expectation is with respect to the random variable encoding the algorithms choices. Considering that the oblivious adversary is deterministic and $\mathcal{OBL}(I) = \text{OPT}(I)$, the cost can be taken out of the expectation and we recover $\frac{\mathbb{E}[\text{RAND}(I)]}{\text{OPT}(I)} \leq \mathbf{c} + \frac{\alpha}{\text{OPT}(I)}$ for this case. It is common to analyse randomized algorithms with respect to the competitiveness against oblivious adversaries and we will stick to this adversary model for the rest of this thesis implicitly.

Randomized online algorithms often achieve lower CRs against $\mathcal{OBL}$ than their deterministic counterparts. As an example, the CR of $2 - \frac{1}{B}$ for deterministic Ski Rental is tight. Using randomization, the CR reduces to $\frac{e}{e-1} \approx 1.58$ [KMMO94]. Intuitively, randomization provides an advantage to ALG in these cases, because it allows to conceal its exact actions to the oblivious adversary, prohibiting it from easily finding the next worst-case request.

2.3 Learning-augmented Online Algorithms

We have seen in the previous section that the CR is a performance measure for online algorithms focusing solely on worst-case input sequences. As with all worst-case analysis approaches, this causes design efforts to concentrate on developing algorithms that are as robust as possible against the worst possible futures to come. It neglects that in practice, often only a small subset of inputs having a specific distribution is encountered. The worst-case input might be incredibly unlikely or even impossible. While this application-agnostic view on algorithms provided by worst-case analysis is considered a major building block of algorithm analysis, theory and applications can profit from aiming at developing and analysing algorithms considering more realistic input sets [Rou21]. Unfortunately, (1) these input distributions are very often unknown and hard to consider in analytical form, and (2) even if such a distribution is given, or a simplifying approximation of it, often the analytical consideration in the algorithm analysis turns out complicated or possibly unfeasible.

Learning-augmented online algorithms introduced in work by Vassilvitskii et al. [MV17, LV21] are able to overcome the classical CR worst-case bounds. The idea is to provide an online algorithm with information about the input distribution in an *implicit* way, enabling it to make better decisions because it has an intuition about what it is likely to receive as future requests.

Analytical consideration of the distributions is not required. The algorithm is informed about an input distribution in the form of predictions, which could stem from a Machine Learning (ML) model. Predictions play a central role in learning-augmented algorithms and they link the field of algorithm design and analysis with the field of ML. ML proved exceptionally good in approximating distributions of data, originally with the goal to extrapolate and generalize on datasets, and hence provides a good starting point for augmenting algorithms with information about the input. In the context of online algorithms, this augmentation can be seen as steering the algorithm into what to consider as likely future requests, decreasing the CR when what is predicted occurs. For example, one could imagine that an algorithm is given the request σ_i for the day, together with a prediction of what a model thinks the request σ_{i+1} will look like. Various forms of predictions are possible.

Of course, a ML model does not always make correct predictions. The presence of this prediction error η (for now not more closely defined) often prohibits an algorithm from just following the prediction blindly in order to attain an acceptable CR. In this framework of learning-augmented online algorithms, we require that the algorithm fulfills the following four properties [MV22]:

1. When the prediction is arbitrarily bad, the CR should match the CR of the previously best known online algorithm (**robustness**)
2. When the prediction is perfect ($\eta = 0$), the performance should be on par with the optimal offline algorithm (**consistency**)
3. There is a smooth transition between robustness and consistency by the algorithm (**smoothness**)

2 Preliminaries

4. Given a suited learnability model, a predictor for the quantity we are using as the prediction is at all learnable (**learnability**)

So we ask a learning-augmented algorithms worst-case performance to lie between the offline optimum and the previously best CR, effectively balanced by the quality of the prediction at hand. Thus, typically, the CR is given as a function of η . We sometimes say an algorithm is α -robust, meaning that for an arbitrarily bad prediction, the algorithm is still α -competitive. Similarly, an algorithm is β -consistent if for $\eta = 0$, its CR is β .

The learnability property makes a statement not about the online algorithm, but about the prediction it is drawing its augmentation from. One would consider a suited learnability model, for example the famous PAC-model [Val84], and examine whether the prediction is learnable. This allows to make statements, for example, if a number of samples polynomial in the request length is enough to learn a predictor with high probability. As this property does not concern the algorithm itself, we do not regard learnability for the remainder of this thesis.

2.3.1 Example: Learning-augmented Ski Rental

Consider again the Ski Rental example. Kumar et al. [PSK18] gave both a deterministic and a randomized learning-augmented online algorithm for the problem. We restate their algorithm for the deterministic case, achieving $1/(1 + \lambda)$ -robustness and $(1 + \lambda)$ -consistency for a user-defined mistrust parameter $\lambda \in (0, 1]$. Consider Algorithm 2.2, provided a single prediction $\hat{p} \in \mathbb{N}$ denoting the models prediction of the length of the ski trip:

Algorithm 2.2 LABRE (LEARNING-AUGMENTED BREAK-EVEN)

Input: $\sigma_i, \hat{p}$
 if $\sigma_i = 0$ or skis bought then return
 if $\hat{p} \geq b$ then
 buy skis on day $\lceil \lambda b \rceil$ ▷ rent until then
 else
 buy skis on day $\lceil \frac{b}{\lambda} \rceil$ ▷ rent until then

The algorithm distinguishes whether the prediction suggests that a longer trip is required, much like OPT does knowing n . LABRE further balances the buying day with a parameter λ , which should be set according to how well the prediction can be trusted. For high trust (low λ), we buy almost immediately when the prediction suggests to do so. Thus, for a low λ and a high η , we should get a bad CR. Exemplarily, we analyse LABRE under the framework of learning-augmentation.

Theorem 2.1 (Competitiveness of Algorithm 2.2 [PSK18]). *For an instance of the Ski Rental problem, a prediction error $\eta := |\hat{p} - n|$, where n denotes the online unknown parameter trip length, and an optimal offline cost OPT , Algorithm 2.2 has a CR of at most*

$$\min \left\{ (1 + \lambda) + \frac{\eta}{(1 - \lambda)\text{OPT}}, 1 + \frac{1}{\lambda} \right\}$$

In particular, Algorithm 2.2 is $(1 + \lambda)$ -consistent and $(1 + 1/\lambda)$ -robust.

Proof. We prove the two bounds separately. As with Algorithm 2.1, the idea is to consider different cases of n . The smoothness-criterion is fulfilled by noting the CRs dependence on η . Following Kumar et al. [PSK18]:

(robustness).

- a. $\hat{p} \geq b$. For $n < \lceil \lambda b \rceil$, both OPT and LABRE pay the same. Hence $n \geq \lceil \lambda b \rceil$. LABRE pays $\lceil \lambda b \rceil - 1 + b$ and the adversary achieves the worst ratio stopping after day $\lceil \lambda b \rceil$, which is what OPT would pay. Thus the CR in this case is $\frac{\lceil \lambda b \rceil - 1 + b}{\lceil \lambda b \rceil} \leq \frac{\lambda b + b}{\lceil \lambda b \rceil} \leq 1 + \frac{b}{\lceil \lambda b \rceil} \leq 1 + \frac{1}{\lambda}$.
- b. $\hat{p} < b$. For $n < b$, OPT and LABRE pay the same. When $b \leq n < \lceil \frac{b}{\lambda} \rceil$, OPT pays b and LABRE pays $n \leq \lceil \frac{b}{\lambda} \rceil - 1$, thus the CR in this case $\frac{\lceil \frac{b}{\lambda} \rceil - 1}{b} \leq \frac{\frac{b}{\lambda}}{b} = \frac{1}{\lambda}$. Finally, for $n \geq \lceil \frac{b}{\lambda} \rceil$, OPT pays b and LABRE pays $\lceil \frac{b}{\lambda} \rceil - 1 + b$, yielding $\frac{\lceil \frac{b}{\lambda} \rceil - 1 + b}{b} \leq \frac{1}{\lambda} + 1$.

(consistency). Now we need to consider the prediction error $\eta := |\hat{p} - n|$. Again:

- a. $\hat{p} \geq b$. As above, only the case $n \geq \lceil \lambda b \rceil$ is interesting. For $n < b$, OPT pays n and we derive $b \leq \hat{p} \leq n + |\hat{p} - n| = \text{OPT} + \eta$. Noting that LABRE pays $\lceil \lambda b \rceil - 1 + b \leq b(1 + \lambda) \leq (\text{OPT} + \eta)(1 + \lambda)$ and dividing by OPT, we obtain a ratio of $(1 + \lambda) + \frac{\eta(1 + \lambda)}{\text{OPT}} \leq (1 + \lambda) + \frac{\eta}{(1 - \lambda)\text{OPT}}$. For $n \geq b$, $b = \text{OPT}$ and thus we can get the ratio $(1 + \lambda)$.
- b. $\hat{p} < b$. Assume $b \leq n < \lceil \frac{b}{\lambda} \rceil$. LABRE pays $n \leq \hat{p} + |\hat{p} - n| \leq b + \eta = \text{OPT} + \eta$. For the case $n \geq \lceil \frac{b}{\lambda} \rceil$, we derive $\eta = |\hat{p} - n| = n - \hat{p} > \lceil \frac{b}{\lambda} \rceil - b \geq (1 - \lambda)\frac{b}{\lambda}$. Thus $\frac{b}{\lambda} \leq \frac{\eta}{1 - \lambda}$. LABRE pays $\lceil \frac{b}{\lambda} \rceil - 1 + \text{OPT} \leq \frac{b}{\lambda} + \text{OPT} \leq \frac{\eta}{1 - \lambda} + \text{OPT}(1 + \lambda)$. This concludes the proof. $\square$

In the CR, scaling η by some factor OPT reflects the view of a normalization of the error. As pointed out by Lykouris et Vassilvitskii [LV21], this normalization is necessary to defend against larger instances naturally having a larger error, without the prediction quality getting worse. Consider doubling the input sequence by appending it to itself. The prediction error will double, although the prediction might be exactly the same. We want our CR to account for that and normalize by OPT. Note that normalizing by the sequence length is not sufficient for most problems, as the input length can be increased without changing OPT [LV21].

Of course, much depends on the concrete definition of the prediction error and we will see some examples of different η in the results of this thesis. In the Related Work section, we provide a further overview of the current topics in the field of learning-augmented algorithms.

2.4 Primal-Dual Methods for Online Algorithm Design

The Primal-Dual method is a design methodology heavily used in the study of approximation algorithms. Both our algorithms presented in Section 4.4 and 5.2 are formulated and analysed in the framework as well. This section provides the necessary background material. For a deeper study in Linear Programming, we refer the reader to the book by Chvátal [CC83], and

2 Preliminaries

to Vazirani [Vaz01] and the survey by Buchbinder et Naor [BN09] for its role in approximation algorithms.

2.4.1 Linear Programming and Duality

A minimization Linear Program (LP) is an optimization problem of the form

Linear Program 2.4.1 (Minimization Standard Form).

$$\begin{array}{ll} \text{minimize} & \sum_{i=1}^n c_i x_i \\ \text{such that} & \sum_{i=1}^n a_{ji} x_i \geq b_j \quad 1 \leq j \leq m \\ & x_i \geq 0 \quad 1 \leq i \leq n \end{array}$$

where a_{ji}, b_j, c_i are arbitrary real-valued parameters and $x_1, \dots, x_n$ are the variables the optimization is over. This minimization LP is in so-called standard form [Vaz01] and using a few tricks like adding variables and flipping inequations, any optimization problem with linear objective under linear constraints can be written down in this form [CLRS09, pp. 850-857].

We call this a primal. Let us introduce a non-negative variable y_j as a multiplicative factor for each primal constraint $1 \leq j \leq m$. Maximizing each y_j , ensuring that for each primal variable x_i the resulting sum of coefficients $\sum_j a_{ji} y_j$ does not exceed c_i , we get a lower-bound on the primal value as $\sum_j b_j y_j$. Let this new linear optimization problem be the dual [Vaz01]:

Linear Program 2.4.2 (Primal-Dual Form).

$\begin{array}{ll} \min & \sum_{i=1}^n c_i x_i \\ \text{s.t.} & \sum_{i=1}^n a_{ji} x_i \geq b_j \quad 1 \leq j \leq m \\ & x_i \geq 0 \quad 1 \leq i \leq n \end{array}$ (Primal)	$\begin{array}{ll} \max & \sum_{i=1}^m b_i y_i \\ \text{s.t.} & \sum_{j=1}^m a_{ji} y_j \leq c_i \quad 1 \leq i \leq n \\ & y_j \geq 0 \quad 1 \leq j \leq m \end{array}$ (Dual)
--	--

By the same method, the dual can be converted back where one obtains the primal program. This relation that the same optimization problem can be described from either a minimization, or a maximization perspective under the same linearity scheme has far-reaching consequences, especially for the design of approximation algorithms. Among others, this is because any feasible solution to the dual program $\mathbf{y} = (y_1, \dots, y_m)$ gives, by construction, a lower bound on the optimal objective cost and hence can be used to obtain an approximation ratio, provided the approximation algorithm also constructs a feasible primal solution $\mathbf{x} = (x_1, \dots, x_n)$. We formalize and unpack this in the following sections.

Further, we introduce the essential terms of fractional and integral solutions. When considering an optimization problem in the framework of Linear Programming, often the variables

encode entities which by the problem definition are only allowed to have discrete values (i.e. integers). An algorithm working on the LP formulation of a problem has to consider that when producing a feasible solution the problem. We call a LP whose variables are relaxed to allow arbitrary positive real-valued solutions the LP-relaxation of the original LP. Further, we refer to a primal/dual solution satisfying these additional discreteness-constraints by the problem definition as an integral solution. We refer to an arbitrary feasible solution to the relaxation as a fractional solution. As the integral solutions form a subspace of all feasible solutions, any optimal fractional solution is also a lower-bound on the integral solutions [BN09].

Finally, we present some LP-theory in the next section, followed by techniques for the design and analysis of approximation algorithms using Linear Programming in Sections 2.4.3 - 2.4.5. As an online algorithm is also an approximation algorithm with respect to the unknownness of the future, the CR describing the worst-case approximation factor over this temporal domain, we get that these techniques naturally translate to online algorithms as well [BN09]. For online algorithms, one would formulate the LP as to have each request represented in the variables, with the proper constraints given by the online problem, and then ensure that after each request σ_i , the online algorithm produces a feasible integral solution for the LP representing the problem for $(\sigma_1, \dots, \sigma_i)$. Of course, the integral solution for past requests cannot be changed.

2.4.2 Duality Theorems and Approximate Complementary Slackness

Here we build up the necessary theory to understand the three main uses of LP-duality in online algorithms called Randomized Rounding 2.4.3, Dual Fitting 2.4.4 and the Primal-Dual Scheme 2.4.5. The aforementioned property that a feasible dual value lower-bounds OPT , in fact lower-bounds any feasible primal solution is also referred to as weak duality and is shown in the following theorem [Vaz01, p. 96].

Theorem 2.2 (Weak Duality). *For a feasible dual solution $\mathbf{y} = (y_1, \dots, y_m)$ and a feasible primal solution $\mathbf{x} = (x_1, \dots, x_n)$, it holds that*

$$\sum_{j=1}^m b_j y_j \leq \sum_{i=1}^n c_i x_i$$

Proof. Using that $\mathbf{y}$ and $\mathbf{x}$ are feasible:

$$\sum_{j=1}^m b_j y_j \leq \sum_{j=1}^m \left(\sum_{i=1}^n a_{ji} x_i \right) y_j = \sum_{j=1}^m \sum_{i=1}^n a_{ji} y_j x_i = \sum_{i=1}^n \left(\sum_{j=1}^m a_{ji} y_j \right) x_i \leq \sum_{i=1}^n c_i x_i$$

□

The theorem gives us a separation of the space of all feasible objective values on the number line into two areas, where the area of all feasible dual objectives contains elements solely smaller, or equal to elements of the primal area. Regarding the transition point of dual

2 Preliminaries

feasible values to primal feasible values, we get the following second theorem of duality [Vaz01, p. 95]:

Theorem 2.3 (Strong Duality). *The primal has a finite optimum solution if and only if the dual has a finite optimum solution. Further, if $\mathbf{x}^* = (x_1^*, \dots, x_n^*)$ and $\mathbf{y}^* = (y_1^*, \dots, y_m^*)$ denotes primal and dual optimum feasible solutions respectively, then*

$$\sum_{j=1}^m b_j y_j^* = \sum_{i=1}^n c_i x_i^*$$

The proof of this statement is more elaborate and we refer to [CC83, pp .58-59] for more details. Strong Duality characterizes the structure of optimal solutions. Finally we introduce another important tool called Approximate Complementary Slackness Conditions [BN09]:

Theorem 2.4 (Approximate Complementary Slackness Conditions). *When for a primal feasible solution $\mathbf{x} = (x_1, \dots, x_n)$ and a dual feasible solution $\mathbf{y} = (y_1, \dots, y_m)$ holds that both*

$$(I) \quad \forall x_i, 1 \leq i \leq n : \quad x_i = 0 \quad \text{or} \quad \frac{c_i}{\alpha} \leq \sum_{j=1}^m a_{ji} y_j \leq c_i$$

$$(II) \quad \forall y_j, 1 \leq j \leq m : \quad y_j = 0 \quad \text{or} \quad b_j \leq \sum_{i=1}^n a_{ji} x_i \leq \beta \cdot b_j$$

for two constants $\alpha \geq 1$ and $\beta \geq 1$, then for the objective values

$$\sum_{i=1}^n c_i x_i \leq \alpha \cdot \beta \cdot \sum_{j=1}^m b_j y_j$$

Proof. Considering the feasibility of the solutions, (I) and (II):

$$\sum_{i=1}^n c_i x_i \leq \alpha \cdot \sum_{i=1}^n \left(\sum_{j=1}^m a_{ji} y_j \right) x_i = \alpha \cdot \sum_{j=1}^m \left(\sum_{i=1}^n a_{ji} x_i \right) y_j \leq \alpha \cdot \beta \cdot \sum_{j=1}^m b_j y_j$$

□

Corollary 2.4.1 (Complementary Slackness Conditions). *When $\alpha = \beta = 1$, using Theorems 2.2 and 2.3, we get that $\mathbf{x}$ and $\mathbf{y}$ are both optimal solutions.*

Conditions (I) and (II) are also referred to as Primal- and Dual Complementary Slackness Conditions respectively [Vaz01, p. 125]. As these conditions can be used to bound primal costs, or even characterize optimal solutions, they are often targeted to fulfill by approximation algorithms.

2.4.3 Randomized Rounding

Working on an optimization problem in the framework of Linear Programming, we naturally face a problem in the gap between fractional and integral solutions. The LP-relaxation

(fractional solutions) of any problem can be solved optimally in weak-polynomial time (e.g. using Ellipsoid Method [GLS81], or Interior Points Methods [Kar84, CLS21]), but this solution is not directly feasible as a problem solution, as the discreteness-constraints might be violated. Our online algorithm has to produce an integral solution.

Thus the probably most straightforward idea is to use such an optimal solver for the relaxation and then round the resulting optimal fractional solution to a feasible integral solution. When not knowing how to round optimally, the rounding has an approximation factor. We refer to Randomized Rounding as a scheme that uses randomization to round the optimal fractional solution to a feasible integral one [BN09]. The fractional values are used as probabilities themselves [Vaz01] and we use this rounding scheme in Section 4.4 for our learning-augmented Parking Permit algorithm to produce an integral solution.

Finally, we show how this allows one to analyse the CR. Consider an arbitrary σ_i and let $P_*^{(i)}$ denote the optimal fractional objective value produced by the solver after processing σ_i . Following Buchbinder et al. [BN09], when one can show that a Randomized Rounding algorithm RAND produces an integral primal solution with objective value $P_{\text{int}}^{(i)}$ such that

$$\mathbb{E} \left[P_{\text{int}}^{(i)} \right] \leq \alpha \cdot P_*^{(i)} \quad \forall i : 1 \leq i \leq n \quad (2.4.1)$$

one immediately gets that RAND is α -competitive against an oblivious adversary by noting that $P_*^{(i)}$ is a lower-bound on the optimal integral solution for $(\sigma_1, \dots, \sigma_i)$.

As a drawback, this method does not make use of the duality-principle and thus of the theory presented in Section 2.4.2. Further, when asking the LP-solver to produce an optimal fractional solution, any Randomized Rounding will naturally have polynomial run time (for polynomially many constraints) [Vaz01]. However, the key issue with this technique is that the majority of the problem solving is handled by the LP-solver and thus disregards any specifics about the problem structure. The rounding can account for that, but offers limited flexibility compared to designing an algorithm tailored to the specific LP at hand [Vaz01].

2.4.4 Method of Dual Fitting

Dual Fitting and the Primal-Dual Scheme 2.4.5 in contrast exploit the specific combinatorial structure of the problem, enabling them to much better adapt to problem variants and generalizations [Vaz01]. As both work on the primal and the dual, they can make use of LP-duality theory. We first introduce the Method of Dual Fitting.

Using Dual Fitting, an online algorithm has to produce a feasible, primal, integral solution, as before at cost $P_{\text{int}}^{(i)}$. This should be based on specifics of the problem at hand and does explicitly not use a general LP-solver. Further, we require the algorithm to produce any dual solution y at cost $D^{(i)}$ ensuring

$$P_{\text{int}}^{(i)} \leq D^{(i)} \quad (2.4.2)$$

One may note that due to Weak- 2.2 and Strong Duality 2.3, a feasible dual solution cannot satisfy this inequality until it is optimal and $P_*^{(i)}$ is integral and indeed this correctly suggests that typically y is not a feasible dual solution.

2 Preliminaries

Before continuing, a few remarks on the application to online algorithms. When processing σ_i , the online algorithm does not change the value of the primal and dual variables encoding the integral solution for $(\sigma_1, \dots, \sigma_{i-1})$, as this will violate the online requirement. Thus only the variables corresponding to the current request are updated. Hence in the analysis of the primal and dual costs, typically the primal and dual objective increase denoted $\Delta P_{\text{int}}^{(i)} := P_{\text{int}}^{(i)} - P_{\text{int}}^{(i-1)}$ and $\Delta D^{(i)} := D^{(i)} - D^{(i-1)}$ is determined. Consequently, noting that $P_{\text{int}}^{(0)} = D^{(0)} = 0$ and summing over the increases, showing $\Delta P_{\text{int}}^{(i)} \leq \Delta D^{(i)}$ is equivalent to showing Eq. (2.4.2) and this is exactly what we do in the proof of Theorem 4.2.

We established that the dual solution $\mathbf{y}$ is infeasible. Now to analyse the CR, the dual must be fitted. For a second step, one has to show that the dual solution produced by the algorithm can be scaled down by a factor α^{-1} such that $\mathbf{y}' := \alpha^{-1}\mathbf{y}$ is a feasible dual solution. Due to linearity, this also scales down $D^{(i)}$ by α^{-1} , creating a feasible dual solution at cost $D'^{(i)} := \alpha^{-1}D^{(i)}$. Using Weak Duality, we determine the algorithm to be α -competitive:

$$P_{\text{int}}^{(i)} \leq D^{(i)} = \alpha \cdot D'^{(i)} \stackrel{2.2}{\leq} \alpha \cdot P_*^{(i)} \leq \alpha \cdot \text{OPT} \quad (2.4.3)$$

As this method does not require a LP solver, the online algorithm can improve on the polynomial runtime.

2.4.5 Primal-Dual Scheme

The Primal-Dual Scheme is the prevalent technique in the use of duality for approximation algorithms, even referred to by Vazirani as the method of choice in this domain [Vaz01, p. 124]. The idea behind the scheme is similar to Dual Fitting in that the online algorithm also has to construct both a primal and a dual solution during runtime. However, the solution process is fundamentally driven by the Approximate Complementary Slackness Conditions (Theorem 2.4) [Vaz01]. For the initial time step $i = 0$ trivially fulfilling these conditions, in each σ_i the online algorithm tries to produce both a feasible primal integral solution and a feasible dual solution satisfying the Slackness Conditions.

Typically this is achieved by continuously increasing the dual variable y_i (say this is corresponding to the current request σ_i) until one of the dual constraints becomes tight. Each primal variable corresponds to a dual constraint by construction of the dual (Section 2.4.1). Hence at that point we know that the Primal Complementary Slackness Condition holds for the primal variable corresponding to the tight dual constraint and that primal variable can be set. The online algorithm then has to fulfill Dual Complementary Slackness by setting the primal variables (integrally), such that the primal constraint corresponding to the raised dual y_i is not violated (or holds for some β factor in the Approximate Slackness version). Setting the primal encodes the actions the algorithm does to serve the request σ_i .

As an example for this procedure, we refer to the learning-augmented deterministic Parking Permit algorithm by Schneider [Sch22] (Section 4.3) and to our learning-augmented algorithm for Online Facility Leasing in Section 5.2.

Following this general recipe, the algorithm constructs a primal feasible integral solution and a dual feasible solution satisfying Approximate Complementary Slackness Conditions.

When one shows that these conditions hold for a give online algorithm (with corresponding α and β as relaxations to the constraints), then the algorithm is $\alpha \cdot \beta$ -competitive as:

$$P_{\text{int}}^{(i)} = \sum_{k=1}^i c_k x_k \stackrel{2.4}{\leq} \alpha \cdot \beta \cdot \sum_{j=1}^m b_j y_j \stackrel{2.2}{\leq} \alpha \cdot \beta \cdot P_*^{(i)} \leq \alpha \cdot \beta \cdot \text{OPT} \quad (2.4.4)$$

2.5 Problem Sets

Here we formally introduce the optimization problems that are relevant to this thesis. Especially noteworthy are Parking Permit and Online Facility Leasing, as we present learning-augmented algorithms for these two problems specifically. Further we state Weighted Set Cover and Online Facility Location as related problems.

2.5.1 Parking Permit

The Online Parking Permit problem introduced by Meyerson [Mey05] is an extension of the previously mentioned Ski Rental in which we are given a sequence, revealed in an online fashion one item at a time, indicating the days we require a parking permit, i.e. the days we are driving to work. This can be interpreted in that due to uncertainties in predicting the future (e.g. bad weather, car damages), we are not able to tell advance whether we will require a parking permit on the next day. A parking permit can be obtained by choosing from a set of K permits, each with different cost C_k and duration D_k , where longer-duration permits tend to be cheaper on a per-day ratio. Parking Permit is thus an instance of rent-or-buy problems. We now define the Parking Permit problem formally.

Problem 2.5.1 (Parking Permit). *For a set of permit types $\mathcal{P} = \{(C_k, D_k)\}_{k=1}^K$ with permit cost $C_k \in \mathbb{Q}^+$ and duration $D_k \in \mathbb{N}$ and a sequence of needs $\sigma \in \{0, 1\}^n$ over a total of n points in time, where $\sigma_i = 1$ denotes the need to access a given resource at time i , we define the Parking Permit problem as finding a set of buying decisions $\mathcal{B} \subseteq \{1, \dots, n\} \times \mathcal{P}$ such that*

$$1. \forall i \in \{1, \dots, n\} : \sigma_i = 1 \implies \exists b = (t, k) \in \mathcal{B} \mid t \leq i < t + D_k \text{ (coverage)}$$

and the total buying cost $\sum_{(t,k) \in \mathcal{B}} C_k$ is minimized.

In the online version of the problem, σ_i is revealed at each time step and when we do so, we require (irreversible) buying decisions for coverage of $\sigma_{1..i}$. Let us introduce some further notation. We refer to the permit types by numbers $1 \dots K$, with a higher-number permit having a larger duration than it's predecessors. By $\mathcal{P}(i)$ we denote the set of permits with their corresponding start of validity that are purchasable for coverage of time step i , that is

$$\mathcal{P}(i) := \bigcup_{k=1}^K \{ (k, t) \mid t \leq i < t + D_k \} \quad (2.5.1)$$

where the set is further restricted in a problem variant called the interval model introduced below.

2 Preliminaries

We don't allow to buy a permit before it is valid. If $\sigma_i = 1$, any algorithm for Parking Permit will buy from $\mathcal{P}(i)$. We can further bound the cardinality of $\mathcal{P}(i)$ by observing that for a permit of type k , at most D_k different purchase times for coverage of i exist, hence $K \leq |\mathcal{P}(i)| \leq \sum_{k=1}^K D_k$.

In the online model, by setting σ to be an uninterrupted sequence of driving days followed by non-driving days for the rest of the sequence and allowing two permits $\mathcal{P} = \{(1, 1), (b, \infty)\}$, we recover the online Ski Rental problem. Additionally, Parking Permit has similarities to the Online Facility Leasing problem [KadHP12, NW13], in that the later is an extension of Parking Permit with an additional geometric problem. It is presented in Section 2.5.4 below.

The offline version of Parking Permit can be solved in $\Theta(n \cdot K)$ time with dynamic programming [Mey05]. For the online problem however, contrary to the Ski Rental problem, no constant CR can be achieved. Meyerson [Mey05] proves a $\Omega(K)$ lower-bound for deterministic and a $\Omega(\log K)$ lower-bound for randomized Parking Permit. The lower-bounds are tight, as is provided a $\mathcal{O}(K)$ -competitive deterministic algorithm and a $\mathcal{O}(\log K)$ -competitive randomized algorithm [Mey05]. The randomized algorithm is based on the Method of Dual Fitting and indeed our learning-augmented algorithm for Parking Permit can be viewed as a direct parameterised extension of this algorithm.

Note that for the following, we refer to the problem in its interval version. In the interval model of Parking Permit, we restrict the space of purchasable permits so that there is exactly one possible permit of each type that could cover a need at a given time. The timeline of the requests is divided, for each permit type k , into a total of $\lceil \frac{n}{D_k} \rceil$ intervals of length D_k . See Fig. 2.1 for a visualization. The lowest level permit is of duration $D_1 = 1$. The second level permit is of duration $D_2 = 3$ and is only available such that no second level intervals overlap. Note that we assume $\forall k, 1 \leq k < K : C_k/D_k > C_{k+1}/D_{k+1}$, such that longer-duration intervals are cheaper on a per-day ratio. Else they can be ignored.

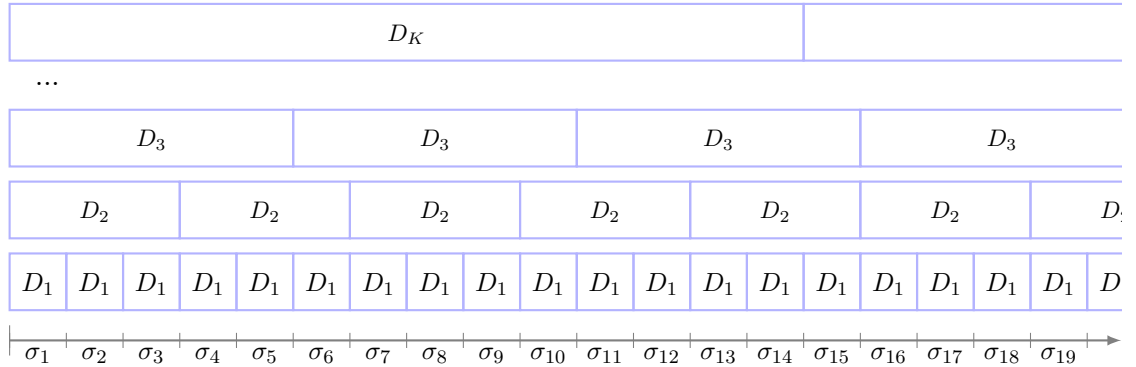


Figure 2.1: Visualization of the interval model in Parking Permit.

The interval model for Parking Permit simplifies the analysis of algorithms and it is easy to proof that when an algorithm is $\alpha(K)$ -competitive for the original version of the problem, it is $2\alpha(K)$ -competitive for the interval version [Mey05]. The optimal permit can be covered by buying the two adjacent intervals of the same type instead, at most doubling the cost. As

there are exactly K available permits at each time step, under the interval model we have that $|\mathcal{P}(i)| = K$ for all $1 \leq i \leq n$. For now on, we will assume the interval version of the problem.

Both algorithms presented for learning-augmented Parking Permit in Chapter 4 are based on the primal-dual formulation and in Linear Program 2.5.1, we give the LP-relaxation and the dual of the problem. We denote by $\mathcal{P}(1..n) := \bigcup_{i=1}^n \mathcal{P}(i)$ the set of all purchasable permits from time 1 to n .

Linear Program 2.5.1 (Parking Permit).

$\begin{aligned} \min \quad & \sum_{(k,t) \in \mathcal{P}(1..n)} C_k x_{k,t} \\ \text{s.t.} \quad & \sum_{(k,t) \in \mathcal{P}(i)} x_{k,t} \geq \sigma_i \quad 1 \leq i \leq n \\ & x_{k,t} \geq 0 \quad (k,t) \in \mathcal{P}(1..n) \end{aligned}$ (Primal)	$\begin{aligned} \max \quad & \sum_{i \in \{1, \dots, n\}} \sigma_i y_i \\ \text{s.t.} \quad & \sum_{t \leq i < t+D_k} y_i \leq C_k \quad (k,t) \in \mathcal{P}(1..n) \\ & y_i \geq 0 \quad 1 \leq i \leq n \end{aligned}$ (Dual)
---	---

Replacing each n with the current observed time step j , we obtain a valid LP for the online version of the problem. The real-valued primal variables $x_{k,t}$ encode some fraction of how much of the corresponding permit is bought. The main primal constraints ensure that every driving day $\sigma_i = 1$ is covered by the sum of the fractional permits. In the Integer Linear Program, we require $x_{k,t} \in \{0, 1\}$. In our algorithm in Section 4.4, we perform Randomized Rounding to produce an integral solution. In the dual problem, the variables y_i can be interpreted as the values of some "packing" of elements into a day, assuming a driving day, while ensuring that over all interval duration, no intervals are overpacked, meaning that its cost would be exceeded. In the deterministic algorithm by Schneider [Sch22], as an example of the design technique Primal-Dual Scheme, the dual formulation plays a central role.

2.5.2 Weighted Set Cover

Set Cover is a classical NP-complete problem [Kar72]. Following Vazirani [Vaz01, p. 15], we have the optimization version of the problem in its weighted form as

Problem 2.5.2 (Weighted Set Cover). *For a universe of elements $\mathcal{U} = \{1, \dots, n\}$, a set of subsets $\mathcal{S} = \{S_1, \dots, S_m\}$ with $S_i \subseteq \mathcal{U}$ for all $1 \leq i \leq m$, and a cost function $w : \mathcal{S} \rightarrow \mathbb{Q}^+$, we define the Weighted Set Cover problem as finding a subset $\mathcal{C} \subseteq \mathcal{S}$ such that*

1. $\bigcup_{S \in \mathcal{C}} S = \mathcal{U}$ (**coverage**)

*and the **total cost** $\sum_{S \in \mathcal{C}} w(S)$ is minimized.*

where in the online version, elements of the universe arrive sequentially in an order unknown to the online algorithm and it irrevocably has to buy subsets as to have $(\sigma_1, \dots, \sigma_i)$ covered. Alon et al. [AAA03] gave the first online algorithm for the problem achieving a CR

2 Preliminaries

of $\mathcal{O}(\log n \log m)$. Their algorithm was also the first to introduce a technique very similar to Dual Fitting in which a feasible fractional primal- and an infeasible dual solution is constructed, subsequently rounding the primal for a feasible integral solution in an online fashion. While our learning-augmented algorithm for Parking Permit uses the same technique combined with Randomized Rounding, Alon et al. give a deterministic rounding scheme and this is responsible for the $\mathcal{O}(\log n)$ -factor in the CR [AAA03, BMS20]. The CR of their online algorithm is almost tight, missing a factor of $(\log \log n + \log \log m)^{-1}$ to the known lower-bound [AAA03].

In its weighted version, Set Cover it is a generalization of the Parking Permit problem. The relation between the problems will be in focus in Section 4.1, where we show how Parking Permit can be transformed into an instance of Weighted Set Cover. Moreover, Weighted Set Cover has a special role in the selection of all problems presented in this thesis. It is prevalent in any of the other problems including Ski Rental, although not always as clear as it might be for Parking Permit after seeing the transformation. The key part is the *coverage* property that any of the problems stated in this thesis has to fulfill. Weighted Set Cover is the most general coverage problem among them. By choosing an appropriate universe $\mathcal{U}$ and subsets S , we can recover any of the other problem formulations.

More generally, Weighted Set Cover is a prevalent example of Coverage-Packing LPs [Vaz01, pp. 109-110]. This term defines a class of primal-dual LP pairs (for ease of notation now given in matrix form) that fulfill

Linear Program 2.5.2 (Coverage-Packing Duality).

$\begin{array}{ll} \min & \mathbf{c}^T \mathbf{x} \\ \text{s.t.} & \mathbf{A}^T \mathbf{x} \geq \mathbf{b} \\ & \mathbf{x} \geq 0 \end{array}$ (Primal, coverage LP)	$\begin{array}{ll} \min & \mathbf{b}^T \mathbf{y} \\ \text{s.t.} & \mathbf{A} \mathbf{y} \leq \mathbf{c} \\ & \mathbf{y} \geq 0 \end{array}$ (Dual, packing LP)
--	---

for non-negative vectors $\mathbf{b}, \mathbf{c}$ and non-negative matrix $\mathbf{A}$. Indeed, this suggests that the interpretation of Linear Program 2.5.1 (Parking Permit) as covering requests and packing elements into permit intervals can be extended to any of the LP formulations presented in this thesis and from now on, we will sometimes refer to the variables as covering requests and packing into intervals respectively. The Coverage-Packing Duality further provides a common scheme along which the LP formulations presented here can be more easily understood and interpreted. Note that for each of the formal problem definitions, we emphasise on the property of coverage to make the connection to Coverage-Packing pairs more clear.

2.5.3 Online Facility Location

Facility Location is a geometric optimization problem asking for the optimal placement of some service providers, from now on called *facilities*. Consider this i.e. to be possible locations to build hospitals on. We constrain the geometric aspect of the problem to metric spaces and further assume that an unlimited amount of service can be provided by each facility, hence from now on implicitly assuming the metric, uncapacitated Facility Location problem.

In the offline version of the problem, we are given a set of possible facility locations and requests in the metric space and an algorithm for Facility Location is asked to find the optimal subset of facilities to open, incurring location-specific opening costs, such that the distances from all requests to the closest open facility and the facility construction costs are minimized. In the hospital example, the requests can be emergency locations for ambulance vehicles, ideally very close to the next open hospital for fast patient care, while considering the cost of building hospitals at every corner as a balance. Once bought, facilities cannot be closed or sold. We now give the formal problem definition.

Problem 2.5.3 (Online Facility Location). *Given a metric space $(\mathcal{M}, d)$ with a finite subset $\mathbb{F} \subseteq \mathcal{M}$ of facility locations, location-dependent construction costs $C : \mathbb{F} \rightarrow \mathbb{Q}^+$ and a request sequence $\sigma \in \mathcal{M}^n$ over n points in space, we define the (metric, uncapacitated) Facility Location problem as finding a set of facility buying decisions $\mathcal{B} \subseteq \{1, \dots, n\} \times \mathbb{F}$ such that*

$$1. \forall i \in \{1, \dots, n\} : \exists b = (t, x) \in \mathcal{B} \mid t \leq i \text{ (coverage)}$$

and the total construction and connection cost to the closest open facility

$$\sum_{(t,x) \in \mathcal{B}} C(x) + \sum_{i=1}^n \min_{(t,x) \in \mathcal{B} \mid t \leq i} d(x, \sigma_i)$$

is minimized.

Guha [GK99, Theorem 3.1] showed the offline problem to be MAXSNP-hard using a reduction from the B -vertex cover problem. Further, they gave an inapproximability bound for polynomial algorithms stating that an approximation ratio less than 1.463 is impossible unless $\text{NP} \subseteq \text{DTIME}(n^{\mathcal{O}(\log \log n)})$. This proof also shows the relation of Facility Location to the Set Cover problem. The offline Facility Location algorithm by Li [Li13] cleverly combines two previous approximation algorithms and achieves the currently best known approximation ratio of 1.488.

In the online version of the problem, the requests are revealed step-by-step where each request has to connect to an open facility before the next one is revealed. The online problem has been introduced by Meyerson [Mey01] and he gave a simple expected $\mathcal{O}(\log n)$ -competitive randomized algorithm, as well as a first non-constant lower-bound on the CR of $\Omega(\log^* n)$ with $\log^* n$ denoting the inverse Ackermann function. Fotakis [Fot08] later improved the results by 1) tightening the analysis by Meyerson [Mey01] establishing his randomized algorithm to be $\mathcal{O}\left(\frac{\log n}{\log \log n}\right)$ -competitive and 2) establishing a $\Omega\left(\frac{\log n}{\log \log n}\right)$ lower-bound, concluding Meyerson's algorithm to be online expected optimal up to constants.

Further, Fotakis gave a deterministic $\Theta\left(\frac{\log n}{\log \log n}\right)$ -competitive algorithm [Fot08]. However, he argues his algorithm to be quite technical and impractical [Fot07], in the same work introducing a simpler deterministic $\mathcal{O}(\log n)$ -competitive algorithm in the primal-dual framework. This algorithm has been restated and reanalysed by Nagarajan et al. [NW13] with a Dual Fitting argument. It is further the basis for a primal-dual online algorithm for the more general Online Facility Leasing problem, which we introduce in the next section.

2.5.4 Online Facility Leasing

Facility Leasing is a straightforward extension of Facility Location additionally defining opening intervals of the facilities. Instead of buying a facility and having it open for an infinite amount of time, facilities have to be leased for which an algorithm has to choose between K different leasing types per facility. Each lease comes with different cost and duration and can be seen as a facility-specific version of Parking Permit, with the lease durations being the same among all facilities. As in Facility Location, the problem lies in opening facility leases in order to connect a set of requests optimally. We define the problem in its metric, uncapacitated version as

Problem 2.5.4 (Online Facility Leasing). *Given a metric space $(\mathcal{M}, d)$ with a finite subset $\mathbb{F} \subseteq \mathcal{M}$ of facility locations, lease durations $D_1, \dots, D_K \in \mathbb{N}$ with location-dependent leasing costs $C : \mathbb{F} \times \{1, \dots, K\} \rightarrow \mathbb{Q}^+$ and a request sequence $\sigma \in \mathcal{M}^n$ over n points in space, we define the (metric, uncapacitated) Facility Leasing problem as finding a set of leasing decisions $\mathcal{L} \subseteq \{1, \dots, n\} \times \{1, \dots, K\} \times \mathbb{F}$ such that*

1. $\forall i \in \{1, \dots, n\} : \exists b = (t, k, x) \in \mathcal{L} \mid t \leq i < t + D_k$ (**coverage**)

and the total leasing and connection cost to the closest open facility

$$\sum_{(t,k,x) \in \mathcal{L}} C(x, k) + \sum_{i=1}^n \min_{(t,k,x) \in \mathcal{L} \mid t \leq i < t + D_k} d(x, \sigma_i)$$

is minimized.

Anthony et al. [AG07] first introduced the offline problem giving an $\mathcal{O}(K)$ -factor approximation algorithm, but it were Nagarajan et Williamson who gave the first constant-factor 3-approximation algorithm [NW13]. Note that the problem is NP-hard, as the Facility Location problem can be recovered with a single lease type of infinite duration.

Combining Meyersons Parking Permit [Mey05] and Online Facility Location [Mey01], one obtains the online version of the problem in which requests are revealed sequentially and are required to be connected to an open lease. Note that our definition here slightly differs from the online definition of the prevalent papers [KadHP12, NW13] in that the later, more generally, allow an arbitrary number of clients (service-needing locations) to arrive with each online request σ_i , while we simplify the model to a single client per request.

Clearly, Facility Leasing extends Parking Permit in that an additional geometric optimization problem has to be solved, adding to the permit buying costs the distance of a request to the closest open facility. Nagarajan et al. provided the first online algorithm for the problem achieving $\mathcal{O}(K \log n)$ -competitiveness [NW13]. The algorithm is an extension of the $\mathcal{O}(\log n)$ -competitive algorithm for Online Facility Location [Fot07] and uses the Primal-Dual Scheme, in combination with a Dual Fitting argument for rescaling the dual variables. We build up on that algorithm of Nagarajan et al. for our algorithm for learning-augmented Facility Leasing. The algorithm and its robustness-analysis are presented in Chapter 5.

Later, Kling et al. [KadHP12] improved on the result by providing an $\mathcal{O}(K \log D_{max})$ -competitive algorithm, where D_{max} denotes the maximal lease duration, thus removing the

dependency on time. Their algorithm is based on a primal-dual 3-approximation algorithm to Facility Location presented by Jain et Vazirani [JV01]. Note that this is not a contradiction to the $\Omega\left(\frac{\log n}{\log \log n}\right)$ lower-bound on Facility Location by Fotakis [Fot08]. As highlighted by Kling et al. [KadHP12], the Facility Location variant by Fotakis further restricts the facility construction as to require at least one request on that location, a restriction that is not in place for the Online Facility Leasing problem. Tight lower-bounds for Online Facility Leasing remain an open question.

We provide a formulation of the LP-relaxation of the Online Facility Leasing problem below. By $\mathcal{F}(i)$ we denote the set of facility intervals with their corresponding start of validity that are purchasable for coverage of time step i , formally

$$\mathcal{F}(i) := \bigcup_{k=1}^K \{ (x, k, t) \mid x \in \mathbb{F}, t \leq i < t + D_k \} \quad (2.5.2)$$

where we often shorten the notation for a specific interval by $s := (x, k, t)$ with s_x denoting the facility location, s_k denoting the lease type and s_t denoting the lease intervals start time. Further, for the rest of the thesis, we shorten the leasing cost notation for some interval s by $C_s := C(s_x, s_k)$.

The following LP formulation of Online Facility Leasing is due to [KadHP12, NW13]. As in Linear Program 2.5.1, $\mathcal{F}(1..n) := \bigcup_{i=1}^n \mathcal{F}(i)$ denotes the set of all purchasable facility intervals from time 1 to n .

Linear Program 2.5.3 (Online Facility Leasing).

$\begin{aligned} \min \quad & \sum_{s \in \mathcal{F}(1..n)} C_s b_s + \sum_{i=1}^n \sum_{s \in \mathcal{F}(i)} d(\sigma_i, s_x) c_{i,s} \\ \text{s.t.} \quad & \sum_{s \in \mathcal{F}(i)} c_{i,s} \geq 1 \quad 1 \leq i \leq n \\ & b_s \geq c_{i,s} \quad 1 \leq i \leq n, s \in \mathcal{F}(1..n) \\ & b_s \geq 0 \quad s \in \mathcal{F}(1..n) \\ & c_{i,s} \geq 0 \quad 1 \leq i \leq n, s \in \mathcal{F}(1..n) \end{aligned}$ (Primal)
$\begin{aligned} \max \quad & \sum_{i=1}^n d_i \\ \text{s.t.} \quad & \sum_{s_t \leq i < s_t + D_{s_k}} \omega_{i,s} \leq C_s \quad s \in \mathcal{F}(1..n) \\ & d_i - \omega_{i,s} \leq d(\sigma_i, s_x) \quad 1 \leq i \leq n, s \in \mathcal{F}(1..n) \\ & d_i \geq 0 \quad 1 \leq i \leq n \\ & \omega_{i,s} \geq 0 \quad 1 \leq i \leq n, s \in \mathcal{F}(1..n) \end{aligned}$ (Dual)

2 Preliminaries

The primal variables can be distinguished in b_s (buy), indicating a purchase of the facility lease, and $c_{i,s}$ (connect), denoting a connection of the request σ_i to (open) facility interval s . As the main primal constraint, among all intervals available at time i , the connection has to sum to at least value 1. Further, the second constraint specifies that in order to connect to an interval s , the interval has to be bought.

For the dual program, we again refer to the view of packing as presented in Section 2.5.2, but we do not have a good interpretation of the variables. Only the duals d_i corresponding to a request σ_i count for the objective, the $\omega_{i,s}$ corresponding to a request-interval connection are specified by the constraint $d_i - d(\sigma_i, s_x) \leq \omega_{i,s}$. A day d_i should pack (bid) as much as possible, ensuring that it packs more than $d(\sigma_i, s_x)$ only if the corresponding $\omega_{i,s}$ is raised, meaning a connection is established, but does not overbid the difference. The request-interval duals $\omega_{i,s}$ are constrained to not pack more into a facility interval than the interval leasing costs.

Problem Simplifications

As a last remark to Online Facility Leasing, we argue about simplifications to the model that prove simplifying and advantageous in the analysis. Note that for intervals increasing in length, exactly as in Parking Permit, we assume the intervals to be cheaper on a per-day ratio, else they can be ignored. They would not be part of any optimal schedule, as one could always buy consecutive cheaper intervals instead. Secondly, again exactly as in Parking Permit, we consider the model in the interval version. This forbids the existence of overlapping schedules of intervals of the same type per facility and effectively reduces the number of leasable intervals per day to $|\mathcal{F}(i)| = K \cdot |\mathbb{F}|$. As for Parking Permit, it can be shown that this at most doubles the cost of the algorithm [AG07, Lemma 2], hence it will asymptotically be irrelevant to the CR.

Further, from now on, we assume all lease durations $D_1, \dots, D_K$ to be powers of two. This at most increases the CR by a factor of 2. The idea is that all lease durations in the original model can be rounded up to the next larger power of 2, still providing a feasible solution, while a solution to the original instance can be reconstructed by buying two consecutive intervals of the same type instead. Kling et al. give the formal proof [KadHP12, Lemma 1].

3 Related Work

In the last chapter, we stated the fundamentals of Competitive Analysis, LP-duality, the idea of learning-augmentation with an example, and finally the optimization problems that are relevant to this thesis. For the problems, we gave upper-bounds and lower-bounds on the competitiveness where available, limited to the pure online algorithms. In this chapter, we extend on the overview of current results by considering the progress in the field of learning-augmented algorithms.

The idea of learning-augmented online algorithms has been introduced by work of Vassilvitskii et Medina for the problem of Revenue Optimization [MV17] and Vassilvitskii et Lykouris for Caching [LV21]. The problem of Caching is an important and well-understood online problem (see [BEY05, 32 ff., 44 ff.] for an introduction) and for a cache size k , they prove their prediction-augmented Marking algorithm (using an ϵ -accurate predictor) to be $2 \cdot \min \{1 + 2S_l(\epsilon), 2H_k\}$ -competitive [LV21, Theorem 3.3]. H_k denotes the k -th Harmonic number and $S_l(\epsilon)$ denotes the *spread*, a technical quantity measuring the growth of a loss function l . They use the prediction of when the requested page will be requested again. For more details on ϵ -accurate predictors and *spread*, we refer to the paper. Their paper is especially significant as it introduced the framework of learning-augmentation in the form we have seen in Section 2.3 and improved on the CR lower-bound known for randomized online Caching. Further learning-augmented algorithms for Caching have been proposed, simplifying and improving the results by combining two algorithms [Wei20], or extending to weighted versions of the problem [BCK⁺].

Another important online problem is k -Server introduced by Manasse et al. [MMS90] where the task is to move up to k -Servers in a metric space as to cover requests revealed within that space under minimal server moving cost. An open problem is whether there exists a (classical) deterministic online algorithm with a CR of k for every metric space (k -Server conjecture [BEY05, p. 152]). In light of learning-augmentation, Antoniadis et al. [ACE⁺20] consider MTS (Metrical Task Systems), a very general model in which k -Server can be retrieved as a special case. They prove the existence of learning-augmented algorithms for MTS, but do not provide a concrete adaptation for k -Server. Lindermayr et al. derived this to produce a 9-consistent and $9k$ -robust algorithm for k -Server on the line metric space [LMS21]. As a technique, Antoniadis et al. use the idea of cleverly combining multiple online algorithms into a single algorithm, achieving a constant-factor increase of the smallest CR among the algorithms. This is a common scheme in learning-augmented algorithm design, allowing to run robust (classical) online algorithms and (possibly unbounded) prediction-augmented algorithms in parallel, achieving robustness and consistency guarantees. The work by Lindermayr et al. [LMS21] improved on the CR obtainable from the MTS generalization for the special case of k -Server on the line. In their prediction model, a request is given a prediction of which of the k servers to move for coverage. For two servers on the line, their learning-

3 Related Work

augmented version of the `DOUBLECOVERAGE` algorithm [BEY05, p. 155] achieves a CR of $\min \left\{ (1 + \lambda) \left(1 + \frac{FTP - OPT}{OPT} \right), 1 + \frac{1}{\lambda} \right\}$, given a parameter $\lambda \in [0, 1]$ and `FTP` as the cost of following the prediction blindly. We build up on that prediction error definition for our Competitive Analysis of learning-augmented Parking Permit and Online Facility Leasing.

The framework of learning-augmented algorithms has also quickly been applied to other online problems like Ski Rental [GP19, PSK18], Scheduling [LLMV, PSK18], Online Matching [ACI21], or Online Network Design problems like Online Steiner Tree [XM21] and Online Facility Location [APT21, JLL⁺21, FGGP21, ACL⁺21]. Bamas et al. provide a general application of the scheme to augment the Primal-Dual method [BMS20], where among others a learning-augmented version of Weighted Set Cover is provided.

Lastly, due to the similarity of Online Facility Leasing to Online Facility Location, we provide results for learning-augmented Facility Location specifically. Fotakis gave the best prediction-augmented CR, but had to withdraw his result due to an error in the main theorem [FGGP21]. A lower-bound theorem by Jiang et al. further contradicts an assumption by Fotakis [JLL⁺21]. Jiang et al.'s algorithm is an extension of the simple expected $\Theta\left(\frac{\log n}{\log \log n}\right)$ -competitive algorithm by Meyerson [Mey01] with predictions. As a prediction, each request gets the location of the facility it should connect to. The basic idea of the algorithm is to first run the classical Meyerson algorithm, creating a budget (of costs) that the prediction step can use to construct further facilities close to the predicted area. For a prediction error η_∞ denoting the maximal distance between any predicted facility location and optimal location, Jiang's algorithm achieves a CR of $\mathcal{O}\left(\min \left\{ \max \left\{ 1, \log \frac{\eta_\infty \cdot n}{OPT} \right\}, \log n \right\}\right)$. As a benefit, the algorithm is arguably simple and does not require any hyperparameters. Unfortunately, we don't think it is easily generalizable to the Online Facility Leasing problem; even extending the classical randomized Meyerson algorithm [Mey01] to obtain a randomized online algorithm for Facility Leasing seems problematic, as the analysis crucially relies on facilities remaining open at all times.

Almanza et al.'s algorithm [ACL⁺21] uses a different prediction model where the algorithm is given a family of sets of possible facility locations $S_1, \dots, S_k \subseteq \mathbb{F}$ beforehand. They show this experimentally to be an easily obtainable prediction [ACL⁺21]. With $\mathcal{S}$ denoting the union of $S_1, \dots, S_k$ and $OPT(\mathcal{S})$ denoting the optimal cost with only opening facilities in $\mathcal{S} \subseteq \mathbb{F}$, their algorithm `WARY` achieves a CR of $\mathcal{O}\left(\min \left\{ \log |\mathcal{S}| \cdot OPT(\mathcal{S}), \frac{\log n}{\log \log n} \cdot OPT \right\}\right)$ [ACL⁺21]. However, the result is limited to facilities with uniform construction cost, contrary to Jiang et al.'s algorithm which allows arbitrary facility construction costs. [JLL⁺21].

Azar et al. [APT21] provide a more general setup on algorithms in metric spaces, of which Online Facility Location is a special case. They introduce an error measure *metric error with outliers* that better accounts for location predictions that are entirely off. For example in Jiang's paper [JLL⁺21], the error-dependent CR was solely based on the maximal distance η_∞ . Their error $\eta = (\Delta, D)$ is given as the minimum cost matching between a subset of predicted vertices and a subset of actual locations D and Δ as the number of outlier points, that do not take part in the matching. There are various choices of Δ and D . When R defines the set of optimal locations that take part in the matching, their learning-augmented algorithm for Online Facility Location has an error-dependent CR of $\mathcal{O}\left(\log(\min \{|R|, \Delta\}) + \frac{D}{OPT}\right)$ [APT21].

As a side note, we mention that the framework of learning-augmentation must not be limited to online algorithm design. Dinitz et al. [DIL⁺21] for example use predictions to reduce the running time of an offline algorithm for weighted bipartite matching. For further study in the field of learning-augmented algorithms, we refer to the Algorithms with Predictions list [LMC23] initiated by Lindermayr and Megow, providing an overview of the publications and resources available for the topic.

To the best of our knowledge, there are currently no known learning-augmented online algorithms for the Parking Permit problem, or the Online Facility Leasing problem. For Parking Permit, we provide both a deterministic- and a randomized learning-augmented algorithm in Chapter 4. In Chapter 5, we present our second main result, a deterministic learning-augmented algorithm for Online Facility Leasing and an analysis of the robustness of the algorithm.

4 Learning-augmented Parking Permit

In this chapter, as our first result, we present a randomized learning-augmented online algorithm for the Parking Permit problem. Beforehand we show how an instance of Parking Permit can be viewed as an instance of Weighted Set Cover. This illustrates the inspiration for our algorithm, which is based on an online Weighted Set Cover algorithm by Bamas et al. [BMS20].

Secondly, as a preparatory step, we define a prediction model for the learning-augmentation in Parking Permit. We argue that there are special characteristics of the problem that require attention when defining a prediction. The model is introduced in Section 4.2.

Before last, we present a deterministic learning-augmented algorithm for Parking Permit introduced by Schneider [Sch22]. Although our randomized version does not inspire much from the deterministic case, both algorithms are given in the Primal-Dual framework and share the need for the prediction model given below. Further, we use the idea of learning-augmentation for the Primal-Dual Scheme presented in Schneider's algorithm [Sch22] in our learning-augmented algorithm for Online Facility Leasing.

Finally, we give the randomized algorithm for Parking Permit and its analysis in Section 4.4.

4.1 Parking Permit as Weighted Set Cover

We note that Parking Permit is a special case of the general Weighted Set Cover problem. Take an instance of Parking Permit $(\mathcal{P}, \sigma)$ in the interval version. Using the following procedure, we obtain an instance of Weighted Set Cover:

1. Set the universe $\mathcal{U} = \{i \in \{1, \dots, n\} \mid \sigma_i = 1\}$ to be the set of indices of the request days that require a permit.
2. Set $\mathcal{S}$ to be the set of all purchasable permits encoded by the indices of their duration. For a permit starting at time step t and lasting for D_k times, this is

$$\mathcal{S} = \bigcup_{i=1}^n \{ \{t, \dots, t + D_k\} \mid (k, t) \in \mathcal{P}(i) \} \quad (4.1.1)$$

3. For each permit $\{t, \dots, t + D_k\} \in \mathcal{S}$, we set the cost function $w(\{t, \dots, t + D_k\}) = C_k$ to be the corresponding permit buying costs.

From the above construction it becomes clear that an optimal solution to this instance is an optimal solution to Parking Permit, as we are looking for the minimal-cost coverage of driving days regarding all available permits.

4.2 Predictions for Parking Permit

Here we introduce the prediction model used for both learning-augmented algorithms presented in the next sections. The model defines a predictor to output a specific permit interval that is suggested for coverage of the current request. However, further assumptions must be made.

Assume we predict any future buying schedule. Predicting the schedule, namely predicting future permit intervals, can be viewed as equivalent to predicting the whole future input sequence and running OPT on it. The predicted buying schedule can not be guaranteed to provide a valid learning-augmentation, as the predicted schedule can falsely leave requests uncovered that in a future iteration of the online algorithm turn out to be driving days. This would cause our CR to additionally be dependent on the number of such holes and is unmeant for. Preventing such invalid buying schedules is hence a first requirement to our prediction model. Secondly, we require a prediction to have a property which we call stability and which is defined below.

Definition 4.2.1 (Stable Prediction). *A prediction $\hat{p} : \{1, \dots, n\} \supseteq A \rightarrow \{1, \dots, K\}$ for an instance of the Parking Permit problem $(\mathcal{P}, \sigma)$ is stable, if $A \supseteq \{1 \leq i \leq n \mid \sigma_i = 1\}$ and*

1. *For $j = \max\{1 \leq k < i \mid \sigma_k = 1\}$, $\hat{p}(j) = \hat{k}$ and $\hat{t}$ as the start of the corresponding interval: $(\sigma_i = 1 \wedge i < \hat{t} + D_{\hat{k}}) \implies \hat{p}(i) = \hat{k}$ (**stability**)*

holds for all $i \in \{1, \dots, n\}$.

Although it might appear as a harsh restriction on the learning-augmented algorithm, we highlight that the stable prediction turns out to be quite realistic and easily obtainable. Consider the property of all driving days being covered by the prediction first. Generating a prediction dynamically when receiving online input guarantees that any day $\sigma_i = 1$ is covered by a prediction, hence possibly benefits from our learning-augmentation. We just *define* the prediction model to give us a covering prediction when we ask it to do so. This circumvents the problem of invalid predicted schedules presented above. From a theoretical viewpoint, the prediction can be just defined like this. From a practical viewpoint, generating predictions dynamically when needed is realistic and can also more easily adapt to changes in input distribution, providing an advantage compared to generating the full prediction up front.

As a second property of the prediction, we require stability. A prediction is stable when once it predicts a permit for a covering, all future predictions for that permit interval do not change. Arguably, this is a rather reasonable requirement. Why should a predictor recommend to buy another permit, when following its prediction from a few time steps ago lead us to be covered already? For non-overlapping solutions, it is clear that the constraint of stability does not hinder the predictor in making optimal predictions. This is because OPT will never cover a driving day twice.

In the interval model of the problem however, there can be optimal solutions with overlapping intervals. Take as an example the instance $(\mathcal{P}, \sigma) = (\{(1, 2), (\frac{3}{2}, 7)\}, (1, 1, 1, 1, 0, 0, 1, 1))$. Nonetheless, we argue that stability does not hurt the predictors ability to predict an optimal covering for these instances. Take an overlapping optimal solution in the interval model and

pick the first overlap. A stable prediction would always predict the left interval for the duration of the overlap. From that point on, take the first cover-needing request, say σ_i , that is solely covered by the right interval. At least one request like this has to exist, because if not, then buying the right interval would have been pointless and hence not part of the optimal covering. By construction, σ_i will be the first request in the right interval for which a prediction is needed and hence an optimal prediction will tell us to buy the right interval, fully consistent with the optimal covering. The same argumentation goes for subsequent overlaps. The only difference to the buying schedule of OPT is that following this prediction, the right interval will be bought after going through the overlap (at time i), instead of possibly when it starts. Neither does this influence the covering, nor the overall cost, hence there is a stable, optimal prediction for every optimal solution.

Stability is an important property for the analysis of the learning-augmented algorithms. It is stated in the definition of a prediction for Parking Permit as a requirement, but we show that any permit-type outputting prediction can be transformed into a stable prediction. Consider the following algorithm:

Algorithm 4.1 MAKE-PREDICTION-STABLE

Input: $\sigma_i = 1, \hat{p}(i) \in \{1 \dots K\}$
 $A \leftarrow [j \in (i - \max_k D_k, i] \mid \sigma_j = 1]$ $\triangleright$ Requires memory $\mathcal{O}(\max_k D_k)$
if $|A| \geq 2$ **and** i **is in interval of** $\hat{p}(A[i - 1])$ **then**
 return $\hat{p}(A[i - 1])$
end if
return $\hat{p}(i)$

Algorithm 4.1 builds a stable prediction for time steps $\sigma_1 \dots \sigma_i$, given a stable prediction for time steps $\sigma_1 \dots \sigma_{i-1}$. The start of the induction can be seen by noting that for $\sigma_1 \dots \sigma_1$ (w.l.o.g assuming $\sigma_1 = 1$), any prediction is stable. For the induction step, we assume $\sigma_i = 1$. The algorithm works by checking the prediction prior to time step i , by induction assumption stable, taking its corresponding interval in the interval model and checking whether i is in that interval. If it is, we redefine $\hat{p}(i)$ to be the previous prediction, achieving stability. If it is not, $\hat{p}(i)$ is the first prediction in a new interval, also stable, and we output it.

Algorithm 4.1 is used as a subroutine in both learning-augmented algorithms for Parking Permit to achieve stability of the permit-type outputting prediction. Why exactly the property is required will become clear in the analysis of the algorithms.

4.3 Deterministic, Learning-augmented Parking Permit

Here we present a deterministic learning-augmented algorithm for the Parking Permit problem proposed by Schneider [Sch22]. The algorithm works on the LP formulation of the problem and uses the Primal-Dual Scheme (Section 2.4.5). We state the algorithm and give an overview of its analysis in the following section.

Algorithm 4.2 DETLAPP (LEARNING-AUGMENTED PARKING PERMIT)

Input: Permit-type outputting Predictor $Predict, \lambda, \sigma_i$

Initialization: $\forall k, t, i : x_{k,t} \leftarrow 0, y_i \leftarrow 0$

if $\sigma_i = 0$ **or** $\sum_{(k,t) \in \mathcal{P}(i)} x_{k,t} \geq 1$ **then return**

$\hat{p}(i) \leftarrow \text{MAKE-PREDICTION-STABLE}(Predict(i))$ ▷ Using Algorithm 4.1

$\mathbb{P}(k) \leftarrow \lambda$ **if** $k = \hat{p}(i)$ **else** 1

increase y_i **and stop immediately when**

(I) $\exists (k, t) \in \mathcal{P}(i) : \sum_{t \leq j < t+D_k} y_j = \mathbb{P}(k) \cdot C_k$

and let X **be the set of permits that satisfies** (I) ▷ $|X| \geq 1$

choose arbitrary $(k, t) \in X$

$x_{k,t} \leftarrow 1$ ▷ Buy permit (k, t)

DETLAPP first checks whether a permit is needed and creates an indicator function $\mathbb{P}(k)$ taking the value λ when k is the predicted permit type, else 1. Following the Primal-Dual Scheme, the dual variable y_i corresponding to the current request is increased until one of the dual constraints from Linear Program 2.5.1 is tight. Noteworthy is the method of learning-augmentation. The algorithm favours a predicted permit type in the increase-step by downscaling the corresponding predicted dual constraint's limit by λ , requiring the dual variables to pay less permit cost for this interval. Out of the set of permits for which the corresponding dual constraints achieved tightness by the increase of y_i , an arbitrary interval is chosen and bought.

4.3.1 Analysis of Detlapp

The analysis is due to Schneider [Sch22]. We give an overview of the analysis. DETLAPP attains the best-possible linear CR of K [Mey05], downscaled by the parameter λ . For a prediction error η as the difference between the optimal solution and a solution by following the prediction blindly, we get a second bound on the CR as $(K\lambda - \lambda + 1) \left(\frac{\eta}{\text{OPT}} + 1 \right)$. This CR tends to 1 for $\eta \rightarrow 0$ and $\lambda = 0$. Note that simply following the prediction leads us to be covered for all driving days, thus creates a feasible solution. Furthermore, predicting an optimal buying schedule is still possible under Definition 4.2.1.

Following the methodology of the Primal-Dual Scheme, we analyse the algorithm by showing that it produces feasible solutions and satisfies the Approximate Complementary Slackness Conditions at all time steps with proper values of α and β . As a main result, we give Theorem 4.1:

Theorem 4.1 (Competitiveness of Algorithm 4.2). *For an instance of the Parking Permit Problem, $\lambda \in [0, 1]$ and a prediction $\hat{p}$, let FTP denote the cost of the solution obtained by following the prediction blindly. Let OPT denote the cost of an optimal offline solution. Then for an error $\eta := \text{FTP} - \text{OPT}$, Algorithm 4.2 has a Competitive Ratio of*

$$\min \left\{ (K\lambda - \lambda + 1) \left(\frac{\eta}{\text{OPT}} + 1 \right), \frac{K}{\lambda} \right\}$$

4.3 Deterministic, Learning-augmented Parking Permit

In particular, Algorithm 4.2 is $(K\lambda - \lambda + 1)$ -consistent and $\frac{K}{\lambda}$ -robust.

Proof. Clearly, DETLAPP constructs a feasible primal integral solution, as its permit variables are initialized at 0 and are set to 1 only if $\sigma_i = 1$ and the request is uncovered. Further, the dual solution is feasible as no dual variable y_i is increased above violating one of the dual constraints. Past dual variables are unaltered. We again separate the proof into robustness and consistency part to show the two bounds separately.

(robustness). Take any non-zero primal variable $x_{k,t}$. We know $x_{k,t} > 0$ only if the corresponding dual constraint was tight. Further, as the interval could have been predicted, but $\mathbb{P}(k) \leq 1$, it holds

$$\lambda C_k \leq \sum_{t \leq j < t + D_k} y_j \leq C_k \quad (4.3.1)$$

thus the Primal Complementary Slackness Conditions hold for $\alpha = \frac{1}{\lambda}$. A dual variable y_i is only increased when $\sigma_i = 1$ and $\sum_{(k,t) \in \mathcal{P}(i)} x_{k,t} < 1$, which causes a primal variable $x_{k,t} = 0$ to be set to 1. Noting non-negativity of the variables, this gives us a lower bound $1 \leq \sum_{(k,t) \in \mathcal{P}(i)} x_{k,t}$ when for the corresponding dual variable $y_i > 0$. Noticing that $x_{k,t} \in \{0, 1\}$ and $|\mathcal{P}(i)| = K$, this gives

$$1 \leq \sum_{(k,t) \in \mathcal{P}(i)} x_{k,t} \leq K \quad (4.3.2)$$

proving the Dual Complementary Slackness Conditions hold and by Theorem 2.4 letting us conclude the CR to be $\frac{K}{\lambda}$.

(consistency). Here we proof an error-dependent CR for a prediction error $\eta := \text{FTP} - \text{OPT}$. FTP denotes the cost of following the prediction blindly. As a general idea, we relate the algorithms cost DETLAPP to FTP, allowing us to substitute the error measure η . Let us fix an arbitrary time n .

As argued in the previous section, FTP produces a feasible solution, even if the prediction is stable. We now consider all predicted permits Λ and note that $\sum_{(\hat{k}, \hat{t}) \in \Lambda} C_{\hat{k}} = \text{FTP}$. This statement critically relies on the prediction being stable - once a permit is predicted, it has to stay predicted until the interval end. This ensures that no new permits can be predicted while FTP would still be covered, allowing us to conclude equality.

The predicted schedule can produce overlap. We eliminate that overlap in the later permit and from now on refer to the predictions by *sections*. More formally, for a predicted permit $(\hat{k}, \hat{t})$, a section $I \subseteq [\hat{t}, \dots, \hat{t} + D_{\hat{k}})$ is the maximal subduration such that $\forall i \in I : \hat{p}(i) = \hat{k}$, including in-between $\sigma_i = 0$ requests. Because our prediction is defined for every driving day, this implies that every $\sigma_i = 1$ is covered by a section. Because our prediction is stable, an interval will always be predicted until the interval-end. We visualize the decomposition of the request sequence into sections in Fig. 4.1 where the yellow boxes indicate the sections.

4 Learning-augmented Parking Permit

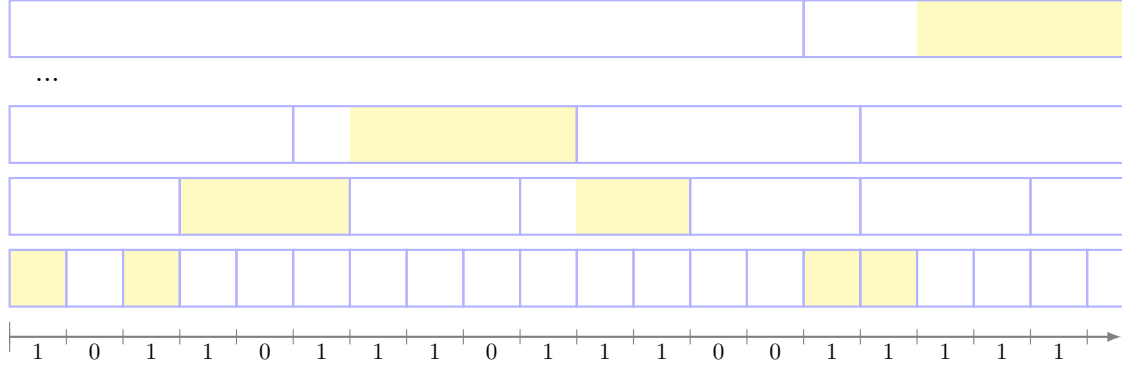


Figure 4.1: The predicted schedule implies sections that cover every driving day.

Let S denote all the permits bought by DETLAPP so far. We bound the cost for $S' := S \setminus \Lambda$, that is the set of all non-predicted intervals bought by DETLAPP. Observing that condition (I) was tight for $(k, t) \in S'$ and that the dual constraint is never violated:

$$\sum_{(k,t) \in S'} C_k = \sum_{(k,t) \in S'} \left(\sum_{t \leq i < t+D_k} y_i \right) \leq \sum_{(k,t) \in \mathcal{P}(1..n) \setminus \Lambda} \left(\sum_{t \leq i < t+D_k} y_i \right) \quad (4.3.3)$$

where the last term denotes the sum of all dual constraints for non-predicted permits. The value of a dual constraint can be interpreted as the bid of that time duration towards buying the corresponding permit. Note that our summation could access duals y_i with $i > n$, but these can be ignored as they have value zero.

For each y_i that is increased by DETLAPP, there is a total of $(K-1)y_i$ increase in the dual constraints corresponding to non-predicted permits in $\mathcal{P}(i)$. Note $|\mathcal{P}(i)| = K$, as we consider the problem in the interval model. Hence over any section I , $\sum_{i \in I} (K-1)y_i$ increase is provided to the other constraints. Summing over all sections $I \in \mathcal{I}$ and their increases, this yields the total value of non-predicted dual constraints as

$$\sum_{(k,t) \in \mathcal{P}(1..n) \setminus \Lambda} \left(\sum_{t \leq i < t+D_k} y_i \right) \leq (K-1) \sum_{I \in \mathcal{I}} \sum_{i \in I} y_i \leq (K-1) \sum_{(\hat{k}, \hat{t}) \in \Lambda} \lambda C_{\hat{k}} \quad (4.3.4)$$

where the last step follows due to $\sum_{i \in I} y_i \leq \lambda C_{\hat{k}}$ for the predicted permit $(\hat{k}, \hat{t})$ corresponding to the section I . That constraint holds because of (I). Combining Eq. (4.3.3) and 4.3.4, we find an upper-bound on the cost of DETLAPP on the subset S' . Further, the cost of DETLAPP of the set $S \cap \Lambda$ can be trivially bounded by FTP. We combine this for

$$\text{DETLAPP} = \text{DETLAPP}_{S'} + \text{DETLAPP}_{S \cap \Lambda} \quad (4.3.5)$$

$$\leq \lambda(K-1) \sum_{(\hat{k}, \hat{t}) \in \Lambda} C_{\hat{k}} + \text{FTP} \quad (4.3.6)$$

$$= \lambda(K-1)\text{FTP} + \text{FTP} = (K\lambda - \lambda + 1)\text{FTP} \quad (4.3.7)$$

$$= (K\lambda - \lambda + 1) + \eta + \text{OPT} \quad (4.3.8)$$

and conclude our proof diving by OPT . $\square$

4.4 A Randomized, Learning-augmented Algorithm for Parking Permit

Here we present a randomized learning-augmented online algorithm for the Parking Permit problem called LAPP . The algorithm is based on Bamas et al.'s algorithm for Online Weighted Set Cover [BMS20] and combines it with a rounding scheme due to Meyerson [Mey05] to omit a $\mathcal{O}(\log n)$ factor in the CR of the original algorithm. This trick results in an expected $\mathcal{O}(\log \frac{K}{\lambda})$ -robust algorithm for Parking Permit, matching the lower-bound on the CR of expected $\Omega(\log K)$ [Mey05] up to a constant. For a prediction error $\eta := \text{FTP} - \text{OPT}$ again as the difference in cost of following the prediction blindly and the optimal cost, LAPP obtains an error-dependent CR of $\mathcal{O}\left(\frac{K}{\lambda(1-K)+K} \left(\frac{\eta}{\text{OPT}} + 1\right)\right)$.

For the analysis, we require the algorithm to take its learning-augmentation from a stable prediction (Definition 4.2.1). We note that Algorithm 4.1, given access to a $\mathcal{O}(\max_k D_k)$ memory on the request sequence, makes any covering, permit-type outputting prediction stable.

Algorithm 4.3 LAPP (LEARNING-AUGMENTED PARKING PERMIT)

Input: Permit-type outputting Predictor Predict , λ, σ_i

Initialization: $\forall k, t, i : x_{k,t} \leftarrow 0, y_i \leftarrow 0$

if $\sigma_i = 0$ **then return**

$\hat{p}(i) \leftarrow \text{MAKE-PREDICTION-STABLE}(\text{Predict}(i))$

$\triangleright$ Using Algorithm 4.1

while $\sum_{(k,t) \in \mathcal{P}(i)} x_{k,t} < 1$ **do**

for all $(k, t) \in \mathcal{P}(i)$ **do**

$x_{k,t} \leftarrow (1 + \frac{1}{C_k})x_{k,t} + \frac{\lambda}{C_k K} + \mathbb{1}\{k = \hat{p}(i)\} \frac{1-\lambda}{C_k}$

end for

$y_i \leftarrow y_i + 1$

end while

if σ_i **is uncovered** **then**

$\triangleright$ Apply randomized rounding scheme [Mey05]

$\tau \sim U(0, 1)$

buy permit interval k **such that**

$\sum_{(j,t) \in \mathcal{P}(i) \mid k+1 \leq j \leq K} x_{j,t} < \tau \leq \sum_{(j,t) \in \mathcal{P}(i) \mid k \leq j \leq K} x_{j,t}$

end if

LAPP takes as input the current online-served request σ_i , the mistrust parameter $\lambda \in [0, 1]$, and a machine-learned predictor Predict (or some oracle of other kind) outputting a predicted permit type for coverage of σ_i when we ask it to do so. This prediction may be based on recent request history, permit-buying habits obtained from data records, or just based on running OPT on a prediction of the future request, given we want to cover day i . The learning-augmentation for the algorithm is stable.

The algorithm works in the Primal-Dual formulation of the problem. For every request $\sigma_i =$

4 Learning-augmented Parking Permit

1, it first ensures that the sum over all primal variables (intervals) purchasable that day satisfies $\sum_{(k,t) \in \mathcal{P}(i)} x_{k,t} \geq 1$. Note that this is a fractional solution to the primal at first. While the condition is not satisfied, the algorithm iteratively increases all primal variables purchasable that day by 1) a $\frac{1}{C_k}$ percentage increase and 2) a fixed additive of $\frac{\lambda}{C_k K} + \mathbb{1}\{k = \hat{p}(i)\} \frac{1-\lambda}{C_k}$. The percentage increase favours cheaper intervals in that their primal variable will grow faster. The additive further favours cheaper permits, but the prediction and λ count as well. When the trust is high (low λ), the additive will almost be zero for all non-predicted intervals. This causes the predicted primal variable to make up most of the sum determining the **while**-condition. For high λ , the predicted term can be neglected and all intervals increase evenly, scaled by $\frac{1}{C_k K}$. Note that the predicted part still increases by a factor K faster than the additive increasing all variables. For $\lambda = 1$, we recover the original algorithm by Meyerson [Mey05].

A dual solution y_i is constructed that counts the number of updates for coverage of that day. It is an infeasible dual solution, as we will see in the analysis and thus we rely on a Dual Fitting argument for the CR. Further, LAPP uses the Randomized Rounding technique to obtain an integral primal solution. This scheme interprets the values of primal variables purchasable that day set above as probabilities and draws a permit according to it. Note that the sum can have values bigger 1. To make up a proper distribution, starting from the longest-duration interval and decreasing the levels step by step, primal variables are considered until they sum up to exactly 1. This can lead to parts of the shorter intervals being ignored by the rounding. The rounding finishes buying a specific interval that covers σ_i .

4.4.1 Analysis of Lapp

In the following, we analyse the algorithm in the interval model. As a main result, we present Theorem 4.2 as a bound on the competitiveness of LAPP. The proof follows the methodology of Dual Fitting and Randomized Rounding. Consider the LP formulation of Parking Permit in Linear Program 2.5.1.

Theorem 4.2 (Competitiveness of Algorithm 4.3). *For an instance of the Parking Permit Problem, $\lambda \in [0, 1]$, and a prediction $\hat{p}$, let FTP denote the cost of the solution obtained by following the prediction blindly. Let OPT denote the cost of an optimal offline solution. Then for an error $\eta := \text{FTP} - \text{OPT}$, Algorithm 4.3 has an expected competitive ratio of*

$$\min \left\{ \mathcal{O} \left(\frac{K}{\lambda(1-K) + K} \right) \left(\frac{\eta}{\text{OPT}} + 1 \right), \mathcal{O} \left(\log \frac{K}{\lambda} \right) \right\}$$

In particular, Algorithm 4.3 is $\mathcal{O} \left(\frac{K}{\lambda(1-K) + K} \right)$ -consistent and $\mathcal{O} \left(\log \frac{K}{\lambda} \right)$ -robust.

Proof. Here we prove the upper bounds on the fractional solution of Algorithm 4.3. This is sufficient to establish the bounds for the randomized, integral solution up to a constant factor. Meyerson [Mey05, Theorem 4.1] provides this rounding scheme and proves that for an $\mathcal{O}(\alpha(K))$ -competitive deterministic fractional algorithm, the scheme gives an expected $\mathcal{O}(\alpha(K))$ -competitive randomized integral algorithm. We first prove the $\mathcal{O} \left(\log \frac{K}{\lambda} \right)$ -robustness.

4.4 A Randomized, Learning-augmented Algorithm for Parking Permit

(robustness). When $P^{(i)}$ denotes the cost of the primal solution after processing σ_i and $D^{(i)}$ the cost of the corresponding dual, we need to upper-bound the ratio of cost increases $\frac{P^{(i)} - P^{(i-1)}}{D^{(i)} - D^{(i-1)}}$. Of course the primal and the dual solutions need to be valid. Note that the costs of P and D are non-decreasing because of the non-negativity constraints in Linear Program 2.5.1.

Let us first find an upper-bound on $P^{(i)} - P^{(i-1)}$. Observing that the dual variable y_i denotes the number of iterations of the **while**-loop, hence the number of updates to the primal variables, we determine that upper-bound as y_i times the maximum primal cost increase in a single iteration.

At first, we establish that for the request σ_i , only those primal variables corresponding to the current purchasable permits $\mathcal{P}(i)$ contribute to a cost increase. For $\sigma_i = 0$, no update to any primal or dual variable is made, hence $P^{(i)} = P^{(i-1)}$ and $D^{(i)} = D^{(i-1)}$. For $\sigma_i = 1$, we update all primal variables in $\mathcal{P}(i)$. After a single iteration of the **while**-loop $j \in \{1, \dots, y_i\}$ with $x_{k,t}^{(i,j)}$ denoting the value of $x_{k,t}$ after that iteration, we have for this iteration's primal cost increase:

$$P^{(i,j)} - P^{(i,j-1)} = \sum_{(k,t) \in \mathcal{P}(i)} C_k \left(x_{k,t}^{(i,j)} - x_{k,t}^{(i,j-1)} \right) \quad (4.4.1)$$

$$= \sum_{(k,t) \in \mathcal{P}(i)} C_k \left(\frac{1}{C_k} x_{k,t}^{(i,j-1)} + \frac{\lambda}{C_k K} + \mathbb{1}\{k = \hat{p}(i)\} \frac{1 - \lambda}{C_k} \right) \quad (4.4.2)$$

Now because in the interval model $|\mathcal{P}(i)| = K$ and the prediction comprises a single permit, this simplifies to

$$= K \frac{\lambda}{K} + 1 - \lambda + \sum_{(k,t) \in \mathcal{P}(i)} x_{k,t}^{(i,j-1)} \quad (4.4.3)$$

$$< 2 \quad (4.4.4)$$

with the loop condition $\sum_{(k,t) \in \mathcal{P}(i)} x_{k,t}^{(i,j-1)} < 1$. Hence for the overall primal increase we have:

$$P^{(i)} - P^{(i-1)} < 2y_i \quad (4.4.5)$$

The loop condition and the variable initialization ensure that the primal solution created by Algorithm 4.3 satisfies all constraints. However, the corresponding dual solution is not valid and needs to be scaled. More precisely, we claim that for any permit $(k, t) \in \mathcal{P}(1 \dots i)$, for the corresponding dual constraint it holds that $\sum_{t \leq i < t + D_k} y_i \leq \mathcal{O}\left(\log \frac{K}{\lambda}\right) C_k$.

Fix one $x_{k,t}$. Our goal is now to relate the primal variables value to the left-hand side dual constraint $\sum_{t \leq i < t + D_k} y_i$, from which we argue that an overpacking of the dual variables has happened. Simplifying the notation from Eq. (4.4.1), we now denote by $x_{k,t}^{(j)}$ the value of a

4 Learning-augmented Parking Permit

primal variable after its j -th update. We observe two natural bounds on any primal variable:

$$(1 + \frac{1}{C_k})x_{k,t}^{(j-1)} + \frac{\lambda}{C_k K} \leq x_{k,t}^{(j)} \leq (1 + \frac{1}{C_k})x_{k,t}^{(j-1)} + \frac{\lambda}{C_k K} + \frac{(1-\lambda)}{C_k} \quad (4.4.6)$$

The upper bound of Eq. (4.4.6) yields

$$x_{k,t}^{(j)} \leq (1 + \frac{1}{C_k})x_{k,t}^{(j-1)} + \frac{1}{C_k} < 1 + \frac{2}{C_k} \quad (4.4.7)$$

since $x_{k,t}^{(j-1)} < 1$ must hold for the update to occur. The lower bound, using the initialization of $x_{k,t}$, leads us to a claim on the growth of $x_{k,t}^{(j)}$:

$$x_{k,t}^{(1)} \geq \frac{\lambda}{C_k K} = \frac{\lambda}{K} \left(1 + \frac{1}{C_k} - 1\right) \quad (4.4.8)$$

$$x_{k,t}^{(2)} \geq (1 + \frac{1}{C_k})\frac{\lambda}{C_k K} + \frac{\lambda}{C_k K} = \frac{\lambda}{K} \left(\frac{2}{C_k} + \frac{1}{C_k^2} + 1 - 1\right) = \frac{\lambda}{K} \left((1 + \frac{1}{C_k})^2 - 1\right) \quad (4.4.9)$$

$$\vdots$$

$$x_{k,t}^{(j)} \geq \frac{\lambda}{K} \left((1 + \frac{1}{C_k})^j - 1\right) \quad (4.4.10)$$

We prove the induction step:

$$x_{k,t}^{(j+1)} \geq (1 + \frac{1}{C_k})\frac{\lambda}{K} \left((1 + \frac{1}{C_k})^j - 1\right) + \frac{\lambda}{C_k K} \quad (4.4.11)$$

$$= \frac{\lambda}{K} \left((1 + \frac{1}{C_k})^j - 1 + \frac{1}{C_k}(1 + \frac{1}{C_k})^j - \frac{1}{C_k} + \frac{1}{C_k}\right) \quad (4.4.12)$$

$$= \frac{\lambda}{K} \left((1 + \frac{1}{C_k})^{j+1} - 1\right) \quad (4.4.13)$$

Using the observation from above that y_i counts the number of updates to a primal variable during processing of σ_i , and further that all updates must happen during the requests $\sigma_t, \dots, \sigma_{t+D_k-1}$, we get $\sum_{t \leq i < t+D_k} y_i$ as a counter of the number of updates to the primal variable. Hence for any point in time:

$$x_{k,t} \geq \frac{\lambda}{K} \left((1 + \frac{1}{C_k})^{\sum_{t \leq i < t+D_k} y_i} - 1 \right) \quad (4.4.14)$$

Without loss of generality, we now assume that for any permit k holds $C_k \geq 1$. Note that the permit costs can always be rescaled without changing the buying schedule of the optimal

4.4 A Randomized, Learning-augmented Algorithm for Parking Permit

solution. Then, using $(1 + 1/C_k)^{C_k} \geq 2$, and using the upper bound from Eq. (4.4.7), we obtain

$$3 > x_{k,t} \geq \frac{\lambda}{K} \left(2^{\frac{1}{C_k} \sum_{t \leq i < t+D_k} y_i} - 1 \right) \quad (4.4.15)$$

This equation can be reformulated to give an upper bound on the dual constraint:

$$\sum_{t \leq i < t+D_k} y_i < \log_2 \left(\frac{3K}{\lambda} + 1 \right) C_k \quad (4.4.16)$$

proving our claim. Hence the dual solution created by Algorithm 4.3 needs a rescaling by $\mathcal{O}(\log \frac{K}{\lambda})^{-1}$ to give a feasible solution. We conclude the proof of robustness by observing $D^{(i)} - D^{(i-1)} = y_i$, scaling y_i by the factor above, and combining this with Eq. (4.4.5) for

$$\frac{P^{(i)} - P^{(i-1)}}{D^{(i)} - D^{(i-1)}} \leq \mathcal{O}\left(\log \frac{K}{\lambda}\right) \quad (4.4.17)$$

(consistency). Recall the definition of the prediction error $\eta := \text{FTP} - \text{OPT}$. For $\sigma_i = 0$, neither FTP or OPT, nor Algorithm 4.3 increase its cost. Hence we consider the case $\sigma_i = 1$ to determine an error-dependent upper bound on the CR. Let $\hat{t}$ be the start of the current interval of the predicted permit type $\hat{p}(i) = \hat{k}$.

We follow the same methodology as in the robustness analysis, namely we upper bound the ratio of the primal solution increase after the j -th **while**-iteration:

$$P^{(i,j)} - P^{(i,j-1)} \leq \sum_{(k,t) \in \mathcal{P}(i)} C_k \left(\frac{1}{C_k} x_{k,t}^{(i,j-1)} + \frac{\lambda}{C_k K} + \mathbb{1}\{k = \hat{p}(i)\} \frac{1-\lambda}{C_k} \right) \quad (4.4.18)$$

$$= \underbrace{(K-1) \frac{\lambda}{K} + \sum_{(k,t) \in \mathcal{P}(i) \setminus (\hat{k}, \hat{t})} x_{k,t}^{(i,j-1)}}_{\text{increase by unpredicted permits}} + \underbrace{x_{\hat{k}, \hat{t}}^{(i,j-1)} + \frac{\lambda}{K} + 1 - \lambda}_{\text{increase by prediction}} \quad (4.4.19)$$

$$\leq \lambda - \frac{\lambda}{K} + 1 + x_{\hat{k}, \hat{t}}^{(i,j-1)} + \frac{\lambda}{K} + 1 - \lambda \quad (4.4.20)$$

$$\leq \left(1 + \frac{\lambda - \frac{\lambda}{K} + 1}{\frac{\lambda}{K} + 1 - \lambda} \right) \left(x_{\hat{k}, \hat{t}}^{(i,j-1)} + \frac{\lambda}{K} + 1 - \lambda \right) \quad (4.4.21)$$

$$= \frac{2K}{\lambda(1-K) + K} \left(x_{\hat{k}, \hat{t}}^{(i,j-1)} + \frac{\lambda}{K} + 1 - \lambda \right) \quad (4.4.22)$$

Now we have bounded the overall fractional increase in a single iteration by the fractional increase due to the prediction $\left(x_{\hat{k}, \hat{t}}^{(i,j-1)} + \frac{\lambda}{K} + 1 - \lambda \right)$ scaled by some factor. Summing over

4 Learning-augmented Parking Permit

all **while**-iterations j and all time steps until now $t \in \{1, \dots, i\}$, this gives us the total fractional cost of Algorithm 4.3 after processing all requests $\sigma_1 \dots \sigma_i$:

$$P = \sum_{t,j} P^{(t,j)} - P^{(t,j-1)} \leq \mathcal{O}\left(\frac{K}{\lambda(1-K) + K}\right) \sum_{t,j} \left(x_{\hat{k},\hat{t}}^{(t,j-1)} + \frac{\lambda}{K} + 1 - \lambda\right) \quad (4.4.23)$$

$$:= \mathcal{O}\left(\frac{K}{\lambda(1-K) + K}\right) P_{\Lambda} \quad (4.4.24)$$

where the last step follows from defining $\Lambda = \{x_{k,t} \mid (k,t) \in \mathcal{P}(j) \wedge k = \hat{p}(j) \wedge \sigma_j = 1 : 1 \leq j \leq i\}$ as the set of all predicted primal variables. P_{Λ} is the fractional cost caused by the predictions. We continue the bounding by looking at the case where the algorithm runs in full-trust mode ($\lambda = 0$), denoted by $\text{LAPP}_{\lambda=0}$. In this case none but the primal variables corresponding to a predicted permit will be updated by the algorithm. We note that because of our definition of the prediction to be a stable prediction, following $\text{LAPP}_{\lambda=0}$, all variables in Λ will be updated and all of these will have a value ≥ 1 after processing a corresponding request. This is because the stable prediction prohibits primal variables to become predicted variables while a request is still covered, eliminating that case that $\text{LAPP}_{\lambda>0}$ could increase predicted variables while $\text{LAPP}_{\lambda=0}$ would leave them untouched at value 0. The requirement of stable predictions ensures that $\text{LAPP}_{\lambda=0}$ updates all variables in Λ :

$$P_{\Lambda} = \sum_{x_{k,t} \in \Lambda} C_k x_{k,t} \stackrel{4.4.15}{\leq} \sum_{x_{k,t} \in \Lambda} 3C_k = 3\text{LAPP}_{\lambda=0} = 3\text{FTP} \quad (4.4.25)$$

The last steps follow by noting that the integer solution $\text{LAPP}_{\lambda=0}$ becomes deterministic and buys exactly the same permits as in Λ , which is equivalent to FTP . We conclude the proof by applying Theorem 4.1 from Meyerson [Mey05] establishing that the rounding scheme gives a randomized algorithm of expected cost $\Theta(\gamma)$ for any fractional solution of cost γ , and rewriting FTP for

$$\mathbb{E}[\text{LAPP}] \leq \mathcal{O}\left(\frac{K}{\lambda(1-K) + K}\right) \left(\frac{\eta}{\text{OPT}} + 1\right) \text{OPT} \quad (4.4.26)$$

□

5 Learning-augmented Facility Leasing

In this chapter we present our second main result on learning-augmented Online Facility Leasing. As an extension of the learning-augmented Parking Permit algorithm from Section 4.3 and Nagarajan et al.'s [NW13] classical online algorithm, we propose a deterministic learning-augmented algorithm for the Online Facility Leasing problem. We begin by introducing our prediction model in Section 5.1. Arguably this is not a straightforward extension of the prediction model for Parking Permit. In the next section, we present the algorithm called LAFL , followed by the analysis of its CR in Section 5.2.1.

Importantly, note that our algorithm is only conjectured to fulfill the properties of learning-augmented online algorithms mentioned in Section 2.3. We only prove the robustness property, ensuring an up to constant factor $\frac{1}{\lambda}$ matching bound on the currently best known online algorithm for the problem. The result is given in Theorem 5.4. Due the limited time of the Masterthesis, we do not prove an error-dependent upper-bound on the CR yet.

5.1 Predictions for Facility Leasing

The prediction model and a corresponding error measure are crucial parts of a learning-augmented algorithm. As we do not have a bound with a corresponding proof for the error-dependent CR of our algorithm so far, our requirements to the prediction model and error measure mentioned here are preliminary.

Noticing that Online Facility Leasing generalizes Parking Permit, it appears natural to extend the predictions provided by the Parking Permit prediction model to include a geometric aspect. For a given request σ_i , we use as learning-augmentation the predictors belief of the optimal covering facility interval $s = (s_x, s_k, s_t)$ with $s_t \leq i < s_t + D_{s_k}$. This is in spirit of the predictions for learning-augmented Online Facility Location presented by Jiang et al. [JLL⁺21], who predict the believed optimal facility location for connecting a request. However, we argue that further requirements on the prediction model are necessary.

We assume it is required to have stability in the prediction, analogue to Definition 4.2.1 for Parking Permit. Once an interval has been predicted, a stable prediction should leave that interval predicted for the rest of the interval duration. We are solving a version of Parking Permit and this required a stable prediction in the analysis of the error-dependent CR. Because of this, we suspect that a requirement like stability appears in the error-dependent analysis of learning-augmented Online Facility Leasing as well. In Parking Permit, stability served the consistency proofs because it ensured that no variables can be updated in a $\lambda > 0$ instance and predicted, without the $\lambda = 0$ instance, equal to FTP, not updating the variable. We argue that an argument similar to the role of stability in Parking Permit might come up in the error-dependent analysis of LAFL .

Unfortunately, requiring stability in the prediction for Online Facility Leasing can prove problematic in the learning-augmentation. Consider the following example: a request appears and its prediction (properly) tells us to connect to a nearby interval $\hat{s}$. That interval thus has to stay predicted; a subsequent request σ_i in the interval duration could be far of and thus will not benefit from the predictor telling it to connect to $\hat{s}$. Optimal schedules have to stay predictable in order to allow constant-factor consistent CRs. To avoid this problem, we suggest to further augment the prediction model in allowing a prediction for σ_i to output multiple intervals. This way, previously predicted intervals can stay predicted until the end of the intervals duration. Among all predicted intervals, we interpret the one closest to σ_i as its learning-augmentation. We give the formal definition.

Definition 5.1.1 (Predictions for Online Facility Leasing). *Let $\mathbb{P}(S)$ denote the power set of a set S . A prediction $\hat{p}$ for an instance of the Online Facility Leasing problem is a function $\hat{p} : \{1, \dots, n\} \rightarrow \mathbb{P}(\mathcal{F}(0..n))$ such that $\hat{p}(i) \in \mathbb{P}(\mathcal{F}(i))$ and $|\hat{p}(i) \setminus \hat{p}(i-1)| \leq 1$ for all $1 \leq i \leq n$. Further, $\hat{p}$ is stable if*

$$1. (\hat{x}, \hat{k}, \hat{t}) \in \hat{p}(i-1) \text{ and } i \in [\hat{t}, \hat{t} + D_{\hat{k}}) \implies (\hat{x}, \hat{k}, \hat{t}) \in \hat{p}(i) \text{ (stability)}$$

for all $i \in \{1, \dots, n\}$.

In an optimal prediction, only the facility intervals that OPT leases will be predicted. This is a zero-error prediction. Any algorithm which follows the prediction in full-trust mode will thus only consider to lease the same intervals as OPT. Now we argue that allowing the prediction for a time step i to additionally output multiple optimal intervals from previous time steps does not hinder a learning-augmented algorithm to buy the optimal schedule in full-trust, when always connecting to the closest facility in $\hat{p}(i)$: For any request σ_i , let $s^* \in \mathcal{F}(i)$ be the interval OPT connects σ_i to. s^* will be in set $\hat{p}(i)$, because the prediction is perfect. It is not possible that another predicted interval from $\hat{p}(i)$ is closer to σ_i than s^* , because all other predicted intervals are optimal (bought by OPT) and OPT would not have connect to s^* , if a closer interval was available. Hence our prediction model enables a learning-augmented algorithm to attain the optimal schedule.

Finally, we note that in order to get a stable prediction for Online Facility Leasing from a single facility interval-outputting predictor $Predict(i)$, we keep track of all previously predicted intervals that could cover the current request σ_i and combine them for a set $\hat{p}(i)$. For the algorithm presented in the next section, we assume this is already implemented.

5.2 A Learning-augmented Algorithm for Facility Leasing

Here we present our deterministic, learning-augmented algorithm for Online Facility Leasing called LAFL. For the notation, refer to the problem definition in Section 2.5.4 and to Linear Program 2.5.3. The algorithm is an extension of Nagarajans [NW13] classical $\mathcal{O}(K \log n)$ -competitive algorithm for Online Facility Leasing and combines it with a learning-augmentation of the Primal-Dual Scheme similar to DETLAPP (Section 4.3) that scales down dual constraints corresponding to predicted facility intervals. Subsequently we analyse the CR of the algorithm. We are able to replace the variable n by the longest interval duration D_{max} ,

5.2 A Learning-augmented Algorithm for Facility Leasing

following a trick due to Kling et al. [KadHP12]. Unfortunately, due to time constraints, we only give the robustness proof for the algorithm. For the consistency proof, we highlight a few ideas. We first give the algorithm. For a term A , we from now on denote $\max\{A, 0\}$ as $(A)_+$.

Algorithm 5.1 LAFL (LEARNING-AUGMENTED FACILITY LEASING)

Input: $\sigma_i, \hat{p}(i), \lambda$

Initialization: $X \leftarrow \emptyset, X_k \leftarrow \emptyset, \forall i, s : b_s \leftarrow 0, c_{i,s} \leftarrow 0, d_i \leftarrow 0, \omega_{i,s} \leftarrow 0$

if $i \bmod D_{max} = 1$ **then** $X \leftarrow \emptyset; X_k \leftarrow \emptyset$ ▷ Reduce memory to $\mathcal{O}(D_{max})$

$\overline{X}(i) \leftarrow \mathcal{F}(i) \setminus X(i)$ ▷ Unleased current intervals

$\mathbb{P}(s) \leftarrow \lambda$ **for** $s \in \hat{p}(i)$ **else** 1

increase d_i **and stop immediately when**

(I) $\exists s \in X(i) : \mathbb{P}(s) \cdot d(\sigma_i, s_x) = d_i$

or

(II) $\exists s \in \overline{X}(i) :$

$$\mathbb{P}(s) \cdot C_s = \left(d_i - d(\sigma_i, s_x) \right)_+ + \sum_{j: s_t \leq j < i} \left(\min\{d_j, d(X_{s_k}(i), \sigma_j)\} - d(\sigma_j, s_x) \right)_+$$

and $d_i \geq d(\sigma_i, s_x)$

and let S **be the set of intervals that satisfied (I) or (II)**

choose $s \in S$ **to be the closest prediction to** σ_i , **else arbitrarily**

if $s \in \overline{X}(i)$ **then**

$X \leftarrow X \cup \{s\}; X_{s_k} \leftarrow X_{s_k} \cup \{s\}$

$b_s \leftarrow 1$

▷ lease facility interval s

end if

for all $s \in \mathcal{F}(1 \dots i)$ **do**

$\omega_{i,s} \leftarrow (d_i / (2H_{D_{max}} + 1) - d(\sigma_i, s_x))_+$

▷ Lemma 5.3

end for

$c_{i,s} \leftarrow 1$

▷ connect σ_i to s_x

From a primal-dual perspective, LAFL constructs an integer primal solution using the tightness of inequalities similar to the dual constraints of Linear Program 2.5.3. These are the main equations in (I) and (II). Dual variables are raised until these conditions are met. Although the raising of duals d_i and its relation to the primal is the main technical principle of the algorithm, we find it more helpful to change the perspective a bit to provide a better intuition on the workings of LAFL. From a geometric point of view, starting from the location σ_i , a request's increasing dual d_i can be thought of as an expanding sphere of radius equal to the current dual value. The expansion continues until either an already open facility interval stored in the set $X(i)$ is hit, denoted (I), or the expansion surpassed an unopened interval in $\overline{X}(i)$ by (possibly combined with other request's overshoot) as much as the unopened interval costs in leasing. This case is denoted (II). Initially an interval has to be leased and the first request's dual contributes all to the interval construction cost. For later requests, their attempt to opening a new facility is supported by previous requests whose sphere also includes the new facility's location. For each unopened interval closer to a request than the

5 Learning-augmented Facility Leasing

boundary of its sphere, there is an overshoot that initially is contributed to that facility. This is encoded in the summation over the so-far seen facility interval duration in (II). However, note that the contribution of a request to opening a facility is reduced by the overshoot it contributed to opening another facility of the same type, when opening that other facility succeeded. This is encoded in the $\min\{\cdot\}$ expression in the summation in (II). We more thoroughly explain and prove this in Lemma 5.2.

Note that the learning-augmentation used here effectively reduces the amount of contribution necessary to lease a facility interval by reducing the interval's cost by factor λ . For already open intervals, them being predicted is identical to moving them closer to the current request σ_i , such that the expansion of the sphere can arrive at them faster. As in the learning-augmented algorithms for Parking Permit, we rely on a user-defined hyperparameter $\lambda \in [0, 1]$.

When the dual d_i has been set and the (possibly multiple) intervals satisfying the conditions are determined, we choose one according to our prediction model mentioned in the previous section. That requires us to connect to the closest predicted interval in order to attain optimal scheduling capability, else we choose arbitrarily. Lastly, LAFL integrally updates the primal variables corresponding to the interval leasing b_s and request connection $c_{i,s}$. The second dual variable $\omega_{i,s}$ is set according to a result of Lemma 5.3 establishing the need for a rescaling of the dual solution.

The condition $d_i \geq d(\sigma_i, s_x)$ required in (II) is proposed as a fix to a problem we argue might occur in the Online Facility Leasing algorithm by Nagarajan et al. [NW13]. A lot of the analysis done in [NW13] relies on the claim that when (II) occurs, $(d_i - d(\sigma_i, s_x))_+ > 0$. For the case that multiple intervals of different leasing types become tight in (II) simultaneously, we argue that it is possible that of these intervals, some unpurchased ones are taken to the next request σ_{i+1} where they will be tight in (II) immediately, without increasing d_{i+1} . More exactly, if during σ_i interval s is bought while an interval s' with $s'_k \neq s_k$ is tight and $d(X_{s'_k}(i+1), \sigma_i) \geq d_i$, condition (II) will be fulfilled for s' during σ_{i+1} instantly. This invalidates the claim that always $(d_i - d(\sigma_i, s_x))_+ > 0$ when (II) occurs. We propose to fix this case by adding the condition $d_i \geq d(\sigma_i, s_x)$. This almost gives the non-negativity claim and we show that this condition suffices to retain the results presented in [NW13]. We continue with an analysis of the robustness of LAFL.

5.2.1 Analysis of Lafli

Although our algorithm uses the Primal-Dual Scheme (Section 2.4.5), its analysis is not based on the Approximate Complementary Slackness Conditions. These conditions do not hold as the dual solution created by LAFL is infeasible. Instead, like in the analysis of Dual Fitting (Section 2.4.4), we lower-bound the dual cost by the primal cost and argue that the dual solution must be rescaled. The main result of our analysis is the robustness case presented in Theorem 5.4. The theorem relies on three lemmata which we prove before.

To begin with, we note that because of our simplification of the problem to the interval version and lease durations as a power of two, all lease types end simultaneously after D_{max} time steps. Then both OPT and LAFL start from the same configuration again, hence it is

5.2 A Learning-augmented Algorithm for Facility Leasing

sufficient to consider a duration of D_{max} to analyse the competitiveness. For the rest of the section, we consider the first such timespan $[1, D_{max}]$. We first bound the primal connection costs and buying costs separately.

Lemma 5.1. *For any point in time i , the connection cost of the primal so far can be bounded from above as*

$$\sum_{j \leq i} \sum_{s \in \mathcal{F}(j)} d(\sigma_j, s_x) c_{j,s} \leq \frac{1}{\lambda} \sum_{j \leq i} d_j$$

Proof. For every request σ_j , exactly one $c_{j,s}$ is set to 1. When we do so, we know that s satisfied either (I) or (II). For (I), $d_j = d(\sigma_j, s_x)$ or $d_j/\lambda = d(\sigma_j, s_x)$. For (II), either $(d_j - d(\sigma_j, s_x))_+ > 0$, in which case the lemma is proven, or $d_j = d(\sigma_j, s_x)$ by the above-mentioned extra condition in (II), when a constraint was tight in a previous request already and is still tight for request σ_j . $\square$

Lemma 5.2. *For any point in time i , the buying cost of the primal so far can be bounded from above as*

$$\sum_{s \in \mathcal{F}(1..i)} C_s b_s \leq \frac{K}{\lambda} \sum_{j \leq i} d_j$$

Proof. Central to this proof is the observation that we buy an interval s if and only if its constraint in (II) got tight. We define a bid of a request as the value its corresponding summand adds to the summation of a specific interval in (II). For a given interval s and time step i , a request σ_j 's bid thus is $(\min\{d_j, d(X_{s_k}(i), \sigma_j)\} - d(\sigma_j, s_x))_+$ for $j \in [s_t, i)$ and $(d_j - d(\sigma_j, s_x))_+$ for $j = i$.

Further let an accepted bid of a request be a bid on an interval when that interval is bought, assuming that the bid was non-zero. Note that bids are non-negative because of the $(\cdot)_+$ -function in the summation. With this definition and the requirement of tightness in (II), we can contribute a cost-function increase for buying interval s to the summation of all accepted bids in that time step. Now if s was predicted, the sum of accepted bids only accounts for λC_s , hence for paying C_s when LAFL buys the interval, we charge $\frac{1}{\lambda}$ times the value of the accepted bids. Considering this case and the reduction of each bid by a non-negative distance term, we get an upper bound on an accepted bid value as d_j/λ . The whole setting is visualized as a bipartite graph in Fig. 5.1.

Yellow nodes in the upper row denote intervals that are bought and hence contribute to the primal buying cost. A full edge indicates an accepted bid with the corresponding edge weight as value and a dotted line indicates a bid value of zero. Note that only leased intervals have accepted bids, hence the total buying cost of the primal so far equals the sum of all incoming full edges for all requests. We now combine the general per-bid upper bound d_j/λ with the following claim:

Claim (Claim of Reducing Bids). *Whenever a bid of a request on an interval s is accepted, say at value α , the requests bids on all other unopened intervals of type s_k are reduced by α as well.*

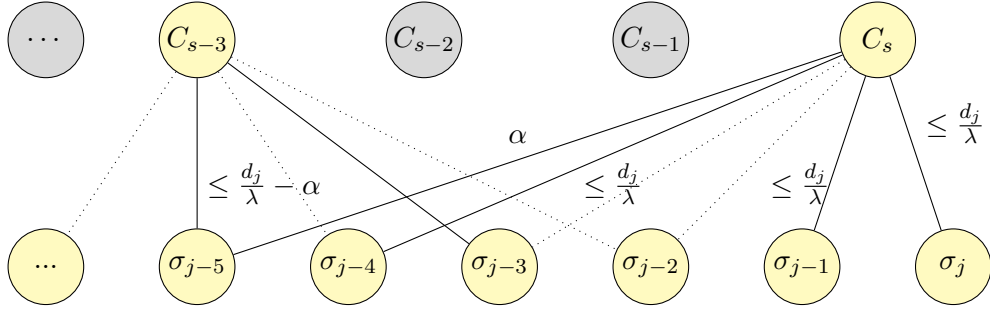


Figure 5.1: Visualization of the primal buying cost charging with accepted bids.

Hence the sum of a requests accepted bids for all intervals of a specific type is be bounded by d_j/λ , serving at most K interval types and summing over all requests, this would prove the lemma. We conclude with proving the Claim of Reducing Bids.

(I) Say that request j was connected to an open interval z with $d_j = \mathbb{P}(z)d(z_x, \sigma_j)$. Starting at the next time step, we consider two unopened intervals s and s' with $s_k = s'_k$ to which j is bidding $(\min \{d_j, d(X_{s_k}(j+1), \sigma_j)\} - d(\sigma_j, s_x))_+$ and $(\min \{d_j, d(X_{s_k}(j+1), \sigma_j)\} - d(\sigma_j, s'_x))_+$ respectively. The bid of j to s becomes accepted during time step $j' > j$, requiring it to be non-zero per definition. The case for s' is completely analog.

- a. $\mathbb{P}(z) = 1$. Say that $\mathbb{P}(s) = 1$. Because the bid is accepted, $d_j = d(z_x, \sigma_j) > d(\sigma_j, s_x)$ and $d(X_{s_k}(j'), \sigma_j) > d(\sigma_j, s_x)$. Hence s must have been closer to σ_j than any open facility of type s_k and also closer than facility z . Sets X and X_{s_k} are updated. Because further $s \in X_{s_k}(j'+1) = X_{s'_k}(j'+1)$, the minimum $\min \{\cdot\}$ of the bids evaluates to $d(s_x, \sigma_j)$. Hence j 's bid to s' reduces to $(\min \{d_j, d(X_{s_k}(j'+1), \sigma_j)\} - d(\sigma_j, s'_x))_+ = (d(s_x, \sigma_j) - d(\sigma_j, s'_x))_+$. Thus it reduced exactly by j 's accepted bid to s with value $(\min \{d_j, d(X_{s_k}(j'), \sigma_j)\} - d(\sigma_j, s_x))_+$. For $\mathbb{P}(s) = \lambda$, the argumentation is the same.
- b. $\mathbb{P}(z) = \lambda$. Say $\mathbb{P}(s) = 1$. When s is bought at j' with an accepted bid, we know $d_j = \lambda d(z, \sigma_j) > d(\sigma_j, s_x)$ and $d(X_{s_k}(j'), \sigma_j) > d(\sigma_j, s_x)$. Hence again s is closer to σ_j than z . The rest follows analogous to a. As above, setting $\mathbb{P}(s) = \lambda$ does not change the non-negativity of accepted bids, or the method of bid reduction, hence we do not need to consider this in a special case.

(II). Let us assume that (II) occurs during processing of request j and that we buy an interval s . Now either the bid of j is non-zero and caused the lease, or the constraint was tight in a previous request already and is just bought now. In that case, due to the second condition in (II), it holds $d_j = d(s_x, \sigma_j)$. Hence j 's bid is zero and thus irrelevant, because we only make a statement about accepted bids.

Again consider another unopened interval s' with $s_k = s'_k$. j 's bid is accepted. We thus know that $d_j > d(\sigma_j, s_x)$ must hold, irrespective of $\mathbb{P}(s)$. Further, no open facility is closer than d_j to σ_j , else we would have (I). The sets X and X_{s_k} are updated accordingly. Consider now time step $j+1$. Then j 's bid to s' is reduced from $(d_j - d(\sigma_j, s'_x))_+$ to $(\min \{d_j, d(X_{s'_k}(j+1), \sigma_j)\} - d(\sigma_j, s'_x))_+$. As above, because $s \in X_{s_k}(j+1) = X_{s'_k}(j+1)$

5.2 A Learning-augmented Algorithm for Facility Leasing

and further s must be the closest open facility to σ_j , the minimum evaluates to $d(s_x, \sigma_j)$, reducing the bid to s' by exactly $(d_j - d(\sigma_j, s_x))_+$. $\square$

As this proof showed us, intuitively, the K factor in the upper-bound on the primal costs (and hence in the robustness, as we will see in Theorem 5.4) stems from a request bidding on all K interval types simultaneously, while its bid is only reduced for intervals of the same type as the one it's bid is accepted for. With Lemma 5.1 and Lemma 5.2, we have successfully upper-bounded the total integral primal costs using a term containing the dual objective.

The following lemma is due to [NW13, Lemma 5.2]. We restate it to fit our notation and to improve readability. We further argue away the dependency on n and consider the case of predictions.

Lemma 5.3. *Let D_{max} be the maximal interval duration. Scaling the dual variables $d_1, \dots, d_n$ set by LAFL with a factor $\theta = (2H_{D_{max}} + 1)^{-1}$ produces a valid solution to the dual program presented in Linear Program 2.5.3.*

Proof. We prove the lemma by showing that the packing inequality

$$C_s \geq \sum_{s_t \leq j < s_t + D_{s_k}} \omega_{j,s} = \sum_{s_t \leq j < s_t + D_{s_k}} (d_j / (2H_{D_{max}} + 1) - d(\sigma_j, s_x))_+$$

holds for any interval $s \in \mathcal{F}(i)$. Thus, in order to fulfill the second inequality $d_i - d(\sigma_i, s_x) \leq \omega_{i,s}$, dual variable d_i needs a scaling of $(2H_{D_{max}} + 1)^{-1}$. The non-negativity constraints are trivially fulfilled.

As we want to bound $\sum_j \omega_{j,s}$, it is sufficient to only consider the $j \in [s_t, s_t + D_{s_k})$ for which $(d_j / (2H_{D_{max}} + 1) - d(\sigma_j, s_x)) > 0$, as the other summands will not contribute to the constraint. We call that subset of requests $\mathcal{J}$ and denote the last request in $\mathcal{J}$ by t^* . Let us now further divide $\mathcal{J}$ into sets A, B with

$$A := \{j \in \mathcal{J} \mid \exists s' \in X_{s_k}(t^*) \text{ such that } d_j \geq d(s', \sigma_j)\} \quad (5.2.1)$$

$$B := \mathcal{J} \setminus A \quad (5.2.2)$$

In Nagarajan et al. [NW13], these correspond to the subsets $\{1, \dots, q\}$ and $\{q + 1, \dots, |S|\}$. Note that the definition of set B implies that $\forall b \in B : d_b < d(X_{s_k}(t^*), \sigma_b)$ and considering (II) at time t^* , this yields

$$C_s \geq (d_{t^*} - d(\sigma_{t^*}, s_x))_+ + \sum_{j: s_t \leq j < t^*} (\min\{d_j, d(X_{s_k}(t^*), \sigma_j)\} - d(\sigma_j, s_x))_+ \quad (5.2.3)$$

$$\geq (d_{t^*} - d(\sigma_{t^*}, s_x))_+ + \sum_{b \in B: b < t^*} (d_b - d(\sigma_b, s_x))_+ \quad (5.2.4)$$

$$= \sum_{b \in B} (d_b - d(\sigma_b, s_x)) \quad (5.2.5)$$

where the first inequality is due to $C_s \geq \mathbb{p}(s) \cdot C_s$. This allows us to easily deal with the case of predictions.

5 Learning-augmented Facility Leasing

As in [NW13], we now order the requests in A first increasingly in the temporal order in which they have fulfilled $d_j \geq d(s', \sigma_j)$ and by decreasing values of $d_j - d(s', \sigma_j)$, in case multiple requests satisfied the condition establishing their belonging to A at the same point in time. Note that this happens when a request causes a facility s' of s_k -type to open and for that reason multiple past requests' duals fulfill the condition for A . Incorrectly for the actual elements, but in favour of readability, we now further refer to the requests in A by numbers $\{1, \dots, |A|\}$. A mapping between the requests and the numbering could be specified.

Fix now an arbitrary $a \in A$. Let a_t be the point in time which causes $d_a \geq d(s', \sigma_a)$ to hold. There might be multiple other requests $a' \leq a$ (in the ordered set A) that share this point in time and we let f be the first such in the ordering. These will all be consecutive predecessors of a , ordered by decreasing $d_{a'} - d(s', \sigma_{a'})$. Note that before finishing processing a_t , all these $f \leq a' \leq a$ satisfy $d_{a'} < d(X_{s_k}(a_t), \sigma_{a'})$, as they not yet have made membership to A . This is to be determined by a_t . All other $1 \leq j < f$ fulfill $d_j \geq d(s', \sigma_j) \geq d(X_{s_k}(a_t), \sigma_j)$ due to their membership in A resulting from a previous request (first criterion of ordering of A). We combine this to derive a criterion that holds due to (II):

$$C_s \geq \mathbb{P}(s) \cdot C_s \quad (5.2.6)$$

$$\geq (d_{a_t} - d(\sigma_{a_t}, s_x))_+ + \sum_{j: s_t \leq j < a_t} (\min \{d_j, d(X_{s_k}(a_t), \sigma_j)\} - d(\sigma_j, s_x))_+ \quad (5.2.7)$$

$$\geq \sum_{j=1}^{f-1} (d(X_{s_k}(a_t), \sigma_j) - d(\sigma_j, s_x))_+ + \sum_{a'=f}^a (d_{a'} - d(\sigma_{a'}, s_x))_+ \quad (5.2.8)$$

$$\geq \sum_{j=1}^{f-1} (d(X_{s_k}(a_t), \sigma_j) - d(\sigma_j, s_x)) + (a - f - 1) \cdot (d_a - d(\sigma_a, s_x)) \quad \triangleright \text{decreasing order} \quad (5.2.9)$$

$$\geq \sum_{j=1}^{f-1} (d_a - d(\sigma_a, s_x) - 2d(\sigma_j, s_x)) + (a - f - 1) \cdot (d_a - d(\sigma_a, s_x)) \quad \triangleright [\text{NW13, Claim 5.3}] \quad (5.2.10)$$

$$\geq -2 \sum_{j=1}^{a-1} d(\sigma_j, s_x) + a \cdot (d_a - d(\sigma_a, s_x)) \quad (5.2.11)$$

Note that the terminology of "connecting" in Nagarajan [NW13, Claim 5.3] refers to requests satisfying the condition of set A , contrary to our use of the term as an actual connection between a request and an open facility being established.

With Eq. (5.2.11), we have derived $|A|$ different constraints for C_s . We continue deriving the packing inequality by adding these constraints, beforehand dividing each 5.2.11 by a and observing that this sum gives a finite Harmonic series $H_{|A|}$ factor for C_s . Combining this further with Eq. (5.2.5):

5.2 A Learning-augmented Algorithm for Facility Leasing

$$(H_{|A|} + 1) \cdot C_s \geq \sum_{a \in A} \left((d_a - d(\sigma_a, s_x)) - \frac{2}{a} \cdot \sum_{j=1}^{a-1} d(\sigma_j, s_x) \right) + \sum_{b \in B} (d_b - d(\sigma_b, s_x)) \quad (5.2.12)$$

$$= \overbrace{\sum_{j \in \mathcal{J}} (d_j - d(\sigma_j, s_x))}^{:= \Delta} - 2 \sum_{a \in A} \left(a^{-1} \sum_{j=1}^{a-1} d(\sigma_j, s_x) \right) \quad (5.2.13)$$

$$= \Delta - 2 \left(\frac{1}{2} \cdot \sum_{j=1}^1 d(\sigma_j, s_x) + \cdots + \frac{1}{|A|} \cdot \sum_{j=1}^{|A|-1} d(\sigma_j, s_x) \right) \quad (5.2.14)$$

$$= \Delta - 2 \left((H_{|A|} - H_1) \cdot d(\sigma_1, s_x) + \cdots + (H_{|A|} - H_{|A|}) \cdot d(\sigma_{|A|}, s_x) \right) \quad (5.2.15)$$

$$= \Delta - 2 \sum_{a \in A} (H_{|A|} - H_a) \cdot d(\sigma_a, s_x) \quad (5.2.16)$$

$$\geq \Delta - \sum_{a \in A} (2H_{|A|} - 1) \cdot d(\sigma_a, s_x) \quad (5.2.17)$$

Using $(d_j - d(\sigma_j, s_x)) = (d_j - 2H_{|A|} \cdot d(\sigma_j, s_x)) + (2H_{|A|} - 1) \cdot d(\sigma_j, s_x)$ and $\mathcal{J} = A \cup B$:

$$\geq \sum_{j \in \mathcal{J}} (d_j - 2H_{|A|} \cdot d(\sigma_j, s_x)) + \sum_{b \in B} (2H_{|A|} - 1) \cdot d(\sigma_b, s_x) \quad (5.2.18)$$

$$\geq \sum_{j \in \mathcal{J}} (d_j - 2(H_{|A|} + 1) \cdot d(\sigma_j, s_x)) \quad (5.2.19)$$

Dividing by $2(H_{|A|} + 1)$, bounding $H_{|A|} \leq H_{D_{max}}$ and considering the properties of $\mathcal{J}$, we have

$$C_s \geq \sum_{j \in \mathcal{J}} (d_j / (2H_{D_{max}} + 1) - d(\sigma_j, s_x))_+ \quad (5.2.20)$$

which concludes the proof. $\square$

Equipped with the preceding lemmata, we are now ready to prove the robustness part of the main theorem of the section regarding the competitiveness of Algorithm 5.1 (LAFL).

Theorem 5.4 (Competitiveness of LAFL). *For an instance of the Online Facility Leasing problem, a prediction $\hat{p}$ and a parameter $\lambda \in [0, 1]$, learning-augmented LAFL achieves a Competitive Ratio of*

$$\mathcal{O}\left(\frac{K}{\lambda} \log D_{max}\right)$$

In particular, LAFL is $\mathcal{O}(\frac{K}{\lambda} \log D_{max})$ -robust.

5 Learning-augmented Facility Leasing

Proof. Combining Lemma 5.1 and Lemma 5.2, we get an upper bound on the primal cost as $P \leq \frac{K+1}{\lambda} \sum_{i=1}^n d_i = \frac{K+1}{\lambda} D$. According to Lemma 5.3, scaling down the dual by $\mathcal{O}((\log D_{\max})^{-1})$ creates a valid dual solution and lets us conclude the robustness case using weak duality. $\square$

Unfortunately, the robustness part alone does not give us guarantees on the advantage of the algorithm using good predictions. Until here we only ensured that our algorithm for learning-augmented Online Facility Leasing does in fact achieve the best-known CR for the problem under our prediction with any error, scaled by a constant factor λ^{-1} . For the rest of this section, we highlight some ideas that might prove useful for the future analysis of the error-dependent CR of LAFL.

The requirement of stability in the prediction was mainly argued because of its role in the analysis of error-dependent CRs for learning-augmented Parking Permit. There it was needed to ensure equality in the cost of a simple strategy of following the prediction blindly (FTP) and the costs caused by the set of all predicted intervals - either fractionally as in LAPP, or integrally as in DETLAPP. Now that we successfully adapted the predictions for Online Facility Leasing for stability as well (Section 5.1), and as it proved useful for the non-geometric problem variant Parking Permit, as a first idea we suggest to apply the same error measure $\eta := \text{FTP} - \text{OPT}$ for analysing LAFL. Note that other error measures are of course possible and many have been developed for online graph problems specifically (i.e. Azar et al.'s *metric error with outliers* [APT21]). It is conceivable that for a successful proof of an error-dependent CR for LAFL, a geometric error measure, possibly combined with an error definition like ours for Parking Permit, will be more useful. Of course, these are preliminary ideas and we suggest to stay with $\eta := \text{FTP} - \text{OPT}$ at first. It has the advantage that naturally FTP relates to the cost of all predicted intervals and thus as a first strategy for a proof, the set of all intervals used by LAFL can be separated into predicted and non-predicted ones, with a relation to the error-measure available for the purchase cost of the predicted ones already. The main difficulty will be to relate the purchased unpredicted intervals to FTP, too. This is similar to the consistency proofs for DETLAPP and LAPP, where the algorithms cost is separated into predicted and unpredicted intervals and the parts are then bounded by FTP. However, the additional (geometric) connection cost has to be considered, making an adaptation of the proofs substantially more difficult.

6 Conclusion

In this thesis we proposed learning-augmented models for the Online Parking Permit problem and its geometric extension, the Online Facility Leasing problem. For each of the two problems, we gave a prediction model that augments an online revealed request σ_i with a prediction $\hat{p}(i)$ that is the predictors believed best covering interval for the time step. The prediction model abstracts from the actual methods used to obtain a prediction, possibly based on Machine Learning or other error-prone oracles. We further introduced a property on the prediction called stability that provides more resilient predictions, simplifying the analysis, and showed that this property can be easily obtained from any interval-outputting prediction with an additional $\mathcal{O}(D_{max})$ memory.

Using the prediction model, we gave a randomized learning-augmented algorithm for the Parking Permit problem called LAPP. We further showed that this algorithm, for a prediction error η as the difference in cost of following the prediction blindly and the optimal offline cost OPT and for a parameter $\lambda \in [0, 1]$, has a CR of $\min \left\{ \mathcal{O}\left(\frac{K}{\lambda(1-K)+K}\right) \left(\frac{\eta}{\text{OPT}} + 1\right), \mathcal{O}\left(\log \frac{K}{\lambda}\right) \right\}$. This yields an improvement over the currently best known randomized online algorithm by Meyerson [Mey05] for low prediction errors and high λ and, to the best of our knowledge, is the first randomized learning-augmented algorithm for the problem.

Based on our prediction model for Parking Permit, we introduced a stable prediction for Online Facility Leasing consisting of multiple intervals simultaneously. For this model, we gave a deterministic learning-augmented algorithm called LAFL and showed this to be $\mathcal{O}\left(\frac{K}{\lambda} \log D_{max}\right)$ -robust. Because of the time-constraints for the thesis, we were not able to prove an error-dependent CR on the algorithm yet. We outlined ideas for the error-measure and the proof and leave this for future work. LAFL extends on ideas used in the deterministic learning-augmentation for Parking Permit presented by Schneider [Sch22] and we conjecture it to be the first learning-augmented algorithm for Online Facility Leasing, but we do not have a bound on the consistency yet.

Lastly, we highlight a few ideas for future work. As a main part of course it remains to prove a consistency bound for LAFL. This achieved, one immediately gets an $\mathcal{O}\left(\frac{\log n}{\lambda}\right)$ -robust deterministic learning-augmented algorithm for Online Facility Leasing. It will be further interesting to see whether the learning-augmentation technique of scaling down the dual constraints for predicted items applied in DETLAPP and LAFL can be extended to other algorithms in the Primal-Dual Scheme. We suspect this to be the case and view this as a method for learning-augmented algorithm design.

Unfortunately, we were not able to remove the dependency on an additional user-defined parameter λ . Learning-augmented algorithms without any hyperparameter exist (i.e. Jiang et al.'s learning-augmented Facility Location [JLL⁺21]) and it remains an open question whether

6 Conclusion

such learning-augmented algorithms for the problems presented here can be designed. We expect that using only the dual-scaling technique for augmentation, this is not possible as λ defines the amount of scaling used there.

Further, we provided prediction models using a novel property called stability. As stability can be easily achieved and proved useful in the error-dependent analysis of both `DETLAPP` and `LAPP`, we argue that it might help in the analysis for other learning-augmented algorithms as well. We showed how it can be extended to Online Facility Leasing and leave it open to use the property for other prediction models as well.

Before last, it remains open to prove error-dependent lower-bounds on the learning-augmented version of the problems, much like Jiang et al. and Fotakis [JLL⁺21, FGGP21] do for learning-augmented Online Facility Location. This guarantees some optimality on the error-dependent part of the CR. Finally, we argue that the predictions used in this thesis can further be evaluated in the context of a learnability-model, for example using the PAC-model.

Bibliography

- [AAA03] Noga Alon, Baruch Awerbuch, and Yossi Azar. The Online Set Cover Problem. In *Proceedings of the Thirty-Fifth Annual ACM Symposium on Theory of Computing*, STOC '03, page 100–105, New York, NY, USA, 2003. Association for Computing Machinery.
- [ACE⁺20] Antonios Antoniadis, Christian Coester, Marek Elias, Adam Polak, and Bertrand Simon. Online metric algorithms with untrusted predictions. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 345–355. PMLR, 2020.
- [ACI21] Anders Aamand, Justin Y Chen, and Piotr Indyk. (Optimal) Online Bipartite Matching with Degree Information, 2021.
- [ACL⁺21] Matteo Almanza, Flavio Chierichetti, Silvio Lattanzi, Alessandro Panconesi, and Giuseppe Re. Online facility location with multiple advice. *Advances in Neural Information Processing Systems*, 34:4661–4673, 2021.
- [AG07] Barbara M Anthony and Anupam Gupta. Infrastructure Leasing Problems. In Matteo Fischetti and David P Williamson, editors, *Integer Programming and Combinatorial Optimization*, pages 424–438, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg.
- [AH12] Susanne Albers and Matthias Hellwig. Semi-online scheduling revisited. *Theoretical Computer Science*, 443:1–9, 2012.
- [APT21] Yossi Azar, Debmalya Panigrahi, and Noam Touitou. Online Graph Algorithms with Predictions, 2021.
- [BCK⁺] Nikhil Bansal, Christian Coester, Ravi Kumar, Manish Purohit, and Erik Vee. Learning-Augmented Weighted Paging. In *Proceedings of the 2022 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 67–89.
- [BEY05] Allan Borodin and Ran El-Yaniv. *Online computation and competitive analysis*. cambridge university press, 2005.
- [BMR⁺20] Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M Ziegler, Jeffrey Wu, Clemens Winter,

Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language Models are Few-Shot Learners, 2020.

- [BMS20] Etienne Bamas, Andreas Maggiori, and Ola Svensson. The Primal-Dual method for Learning Augmented Algorithms. In H Larochelle, M Ranzato, R Hadsell, M F Balcan, and H Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 20083–20094. Curran Associates, Inc., 2020.
- [BN09] Niv Buchbinder and Joseph Naor. The Design of Competitive Online Algorithms via a Primal-Dual Approach. *Found. Trends Theor. Comput. Sci.*, 3(2-3):93–263, 2009.
- [CC83] V Chvátal and V Chvátal. *Linear Programming*. Series of books in the mathematical sciences. W.H. Freeman, 1983.
- [CLRS09] Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. *Introduction to Algorithms, Third Edition*. The MIT Press, 3rd edition, 2009.
- [CLS21] Michael B Cohen, Yin Tat Lee, and Zhao Song. Solving Linear Programs in the Current Matrix Multiplication Time. *J. ACM*, 68(1), 1 2021.
- [DIL⁺21] Michael Dinitz, Sungjin Im, Thomas Lavastida, Benjamin Moseley, and Sergei Vassilvitskii. Faster Matchings via Learned Duals, 2021.
- [FGGP21] Dimitris Fotakis, Evangelia Gergatsouli, Themis Gouleakis, and Nikolas Patrís. Learning Augmented Online Facility Location, 2021.
- [Fot07] Dimitris Fotakis. A primal-dual algorithm for online non-uniform facility location. *Journal of Discrete Algorithms*, 5(1):141–148, 2007.
- [Fot08] Dimitris Fotakis. On the competitive ratio for online facility location. *Algorithmica*, 50(1):1–57, 2008.
- [GK99] Sudipto Guha and Samir Khuller. Greedy strikes back: Improved facility location algorithms. *Journal of algorithms*, 31(1):228–248, 1999.
- [GLS81] M. Grötschel, L. Lovász, and A. Schrijver. The ellipsoid method and its consequences in combinatorial optimization. *Combinatorica*, 1(2):169–197, 1981.
- [GP19] Sreenivas Gollapudi and Debmalaya Panigrahi. Online algorithms for rent-or-buy with expert advice. In *International Conference on Machine Learning*, pages 2319–2327. PMLR, 2019.
- [JLL⁺21] Shaofeng H C. Jiang, Erzhi Liu, You Lyu, Zhihao Gavin Tang, and Yubo Zhang. Online Facility Location with Predictions, 2021.

- [JV01] Kamal Jain and Vijay V Vazirani. Approximation Algorithms for Metric Facility Location and K-Median Problems Using the Primal-Dual Schema and Lagrangian Relaxation. *J. ACM*, 48(2):274–296, 3 2001.
- [KadHP12] Peter Kling, Friedhelm auf der Heide, and Peter Pietrzyk. An algorithm for online facility leasing. In *International Colloquium on Structural Information and Communication Complexity*, pages 61–72. Springer, 2012.
- [Kar72] Richard M Karp. Reducibility among Combinatorial Problems. In Raymond E Miller, James W Thatcher, and Jean D Bohlinger, editors, *Complexity of Computer Computations: Proceedings of a symposium on the Complexity of Computer Computations, held March 20–22, 1972, at the IBM Thomas J. Watson Research Center, Yorktown Heights, New York, and sponsored by the Office of Naval Research, Ma*, pages 85–103. Springer US, Boston, MA, 1972.
- [Kar84] N Karmarkar. A New Polynomial-Time Algorithm for Linear Programming. In *Proceedings of the Sixteenth Annual ACM Symposium on Theory of Computing*, STOC '84, page 302–311, New York, NY, USA, 1984. Association for Computing Machinery.
- [KMMO94] Anna R Karlin, Mark S Manasse, Lyle A McGeoch, and Susan Owicki. Competitive randomized algorithms for nonuniform problems. *Algorithmica*, 11(6):542–571, 1994.
- [KPS⁺18] Ravi Kumar, Manish Purohit, Aaron Schild, Zoya Svitkina, and Erik Vee. Semi-Online Bipartite Matching, 2018.
- [Li13] Shi Li. A 1.488 approximation algorithm for the uncapacitated facility location problem. *Information and Computation*, 222:45–58, 2013.
- [LLMV] Silvio Lattanzi, Thomas Lavastida, Benjamin Moseley, and Sergei Vassilvitskii. Online Scheduling via Learned Weights. In *Proceedings of the 2020 ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1859–1877.
- [LMC23] Alexander Lindermayr, Nicole Megow, and Contributors. Algorithms with Predictions. <https://algorithms-with-predictions.github.io/>, 2023. [Online; accessed 31-January-2023].
- [LMS21] Alexander Lindermayr, Nicole Megow, and Bertrand Simon. Double Coverage with Machine-Learned Advice, 2021.
- [LV21] Thodoris Lykouris and Sergei Vassilvitskii. Competitive Caching with Machine Learned Advice. *Journal of the ACM*, 68(4):1–25, 7 2021.
- [Mey01] Adam Meyerson. Online facility location. In *Proceedings 42nd IEEE Symposium on Foundations of Computer Science*, pages 426–431. IEEE, 2001.

- [Mey05] Adam Meyerson. The parking permit problem. In *Proceedings - Annual IEEE Symposium on Foundations of Computer Science, FOCS*, volume 2005, pages 274–282. IEEE, 2005.
- [MMS90] Mark S Manasse, Lyle A McGeoch, and Daniel D Sleator. Competitive Algorithms for Server Problems. *J. Algorithms*, 11(2):208–230, 5 1990.
- [MN20] Nicole Megow and Lukas Nölke. Online Minimum Cost Matching with Recourse on the Line. In Jarosław Byrka and Raghu Meka, editors, *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2020)*, volume 176 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 37:1–37:16, Dagstuhl, Germany, 2020. Schloss Dagstuhl–Leibniz-Zentrum für Informatik.
- [MSVW16] Nicole Megow, Martin Skutella, José Verschae, and Andreas Wiese. The Power of Recourse for Online MST and TSP. *SIAM Journal on Computing*, 45(3):859–880, 2016.
- [MV17] Andrés Muñoz Medina and Sergei Vassilvitskii. Revenue Optimization with Approximate Bid Predictions, 2017.
- [MV22] Michael Mitzenmacher and Sergei Vassilvitskii. Algorithms with Predictions. *Commun. ACM*, 65(7):33–35, 6 2022.
- [NW13] Chandrashekhar Nagarajan and David P Williamson. Offline and online facility leasing. *Discrete Optimization*, 10(4):361–370, 2013.
- [PSK18] Manish Purohit, Zoya Svitkina, and Ravi Kumar. Improving Online Algorithms via ML Predictions. In S Bengio, H Wallach, H Larochelle, K Grauman, N Cesa-Bianchi, and R Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018.
- [PSTK15] Jigar Patel, Sahil Shah, Priyank Thakkar, and Ketan Kotecha. Predicting stock and stock price index movement using trend deterministic data preparation and machine learning techniques. *Expert systems with applications*, 42(1):259–268, 2015.
- [Rou21] Tim Roughgarden. *Beyond the Worst-Case Analysis of Algorithms*. Cambridge University Press, 2021.
- [Sch22] Florian Schneider. Deterministic Learning-augmented Parking Permit. Personal Communication, 2022.
- [ST85] Daniel D Sleator and Robert E Tarjan. Amortized Efficiency of List Update and Paging Rules. *Commun. ACM*, 28(2):202–208, 2 1985.
- [Val84] Leslie G Valiant. A theory of the learnable. *Communications of the ACM*, 27(11):1134–1142, 1984.

- [Vaz01] Vijay V Vazirani. *Approximation Algorithms*. Springer Berlin, Heidelberg, ü, 2001.
- [VBC⁺19] Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev, and others. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.
- [Wei20] Alexander Wei. Better and Simpler Learning-Augmented Online Caching, 2020.
- [XM21] Chenyang Xu and Benjamin Moseley. Learning-Augmented Algorithms for Online Steiner Tree, 2021.

Eidesstattliche Erklärung

Hiermit versichere ich an Eides statt, dass ich die vorliegende Arbeit im Masterstudiengang Informatik selbstständig verfasst und keine anderen als die angegebenen Hilfsmittel — insbesondere keine im Quellenverzeichnis nicht benannten Internet-Quellen — benutzt habe. Alle Stellen, die wörtlich oder sinngemäß aus Veröffentlichungen entnommen wurden, sind als solche kenntlich gemacht. Ich versichere weiterhin, dass ich die Arbeit vorher nicht in einem anderen Prüfungsverfahren eingereicht habe und die eingereichte schriftliche Fassung der elektronischen Abgabe entspricht.

Hamburg, den 03.02.2023

Marcus Philip Rottschäfer

Veröffentlichung

Ich stimme der Einstellung der Arbeit in die Bibliothek des Fachbereichs Informatik zu.

Hamburg, den 03.02.2023

Marcus Philip Rottschäfer